BlockChain



Mrs. Pallavi Patil
(Computer Engineering)

Module 1.Introduction to Blockchain

1.1 What is a blockchain, Origin of blockchain (cryptographically secure hash functions), Foundation of blockchain: Merkle trees

1.2 Components of blockchain, Block in blockchain, Types: Public, Private, and Consortium, Consensus Protocol, Limitations and Challenges of blockchain



QBank

1. Define blockchain? Compare different types of blockchain (5)
2. what is a Merkle tree? Explain the structure of a Merkle tree(5)
3. State and explain various challenges that occur while implementing blockchain.(10)
4. Differentiate between public, private and consortium blockchain(10)
5. Explain Merkle Tree with help of an example.(10)



History of Blockchain

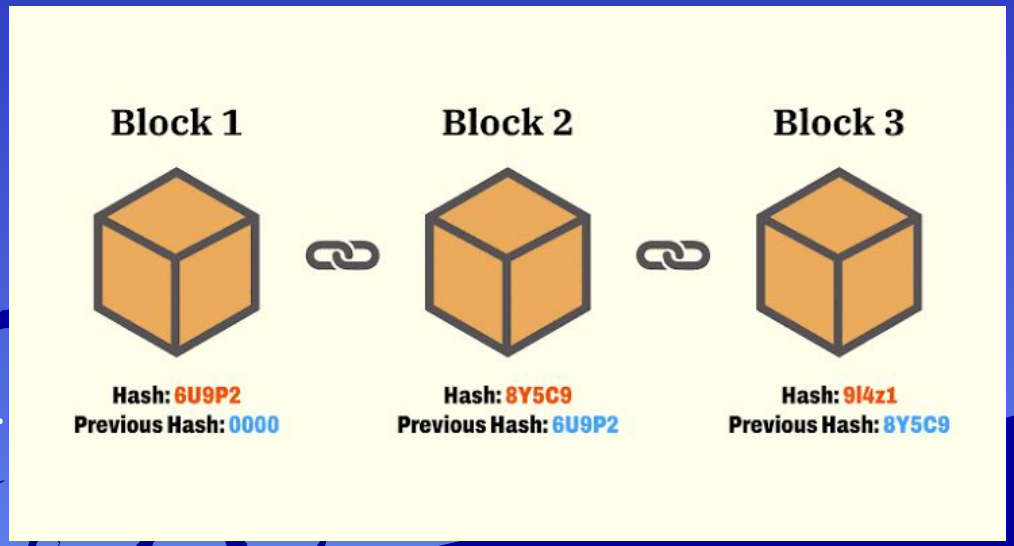
What is Blockchain?

A **blockchain** is a digital ledger (record book) that stores data in **blocks**. These blocks are linked together in a **chain** using cryptography, making the records secure and difficult to alter.

A **blockchain** is a digital ledger (record book) that stores information in a way that is secure, transparent, and difficult to alter. Instead of being stored on a single computer, the data is shared across many computers (called nodes) in a network.

Imagine a notebook shared by thousands of people. Whenever a new transaction is written:

1. The information is stored in a **block**.
2. The block is verified by the network.
3. The block is linked to the previous block.
4. The updated ledger is copied across all computers in the network.



This makes blockchain:

- **Decentralized** – no single person or organization controls it.
- **Transparent** – all participants can view the records.
- **Secure** – records cannot be easily changed.
- **Immutable** – once recorded, data is very difficult to modify.

Origin of blockchain

Early Foundations (1991): The concept of blockchain was first proposed by researchers:

- Stuart Haber & W. Scott Stornetta

They developed a method for timestamping digital documents so they could not be tampered with.

Development (1998–2004): Several researchers proposed digital currency systems and cryptographic techniques, laying the groundwork for blockchain technology.

Birth of Modern Blockchain (2008): In 2008, a person or group

- Satoshi Nakamoto

published the paper **Bitcoin: A Peer-to-Peer Electronic Cash System**, describing a decentralized digital currency called Bitcoin.

First Blockchain Network (2009)

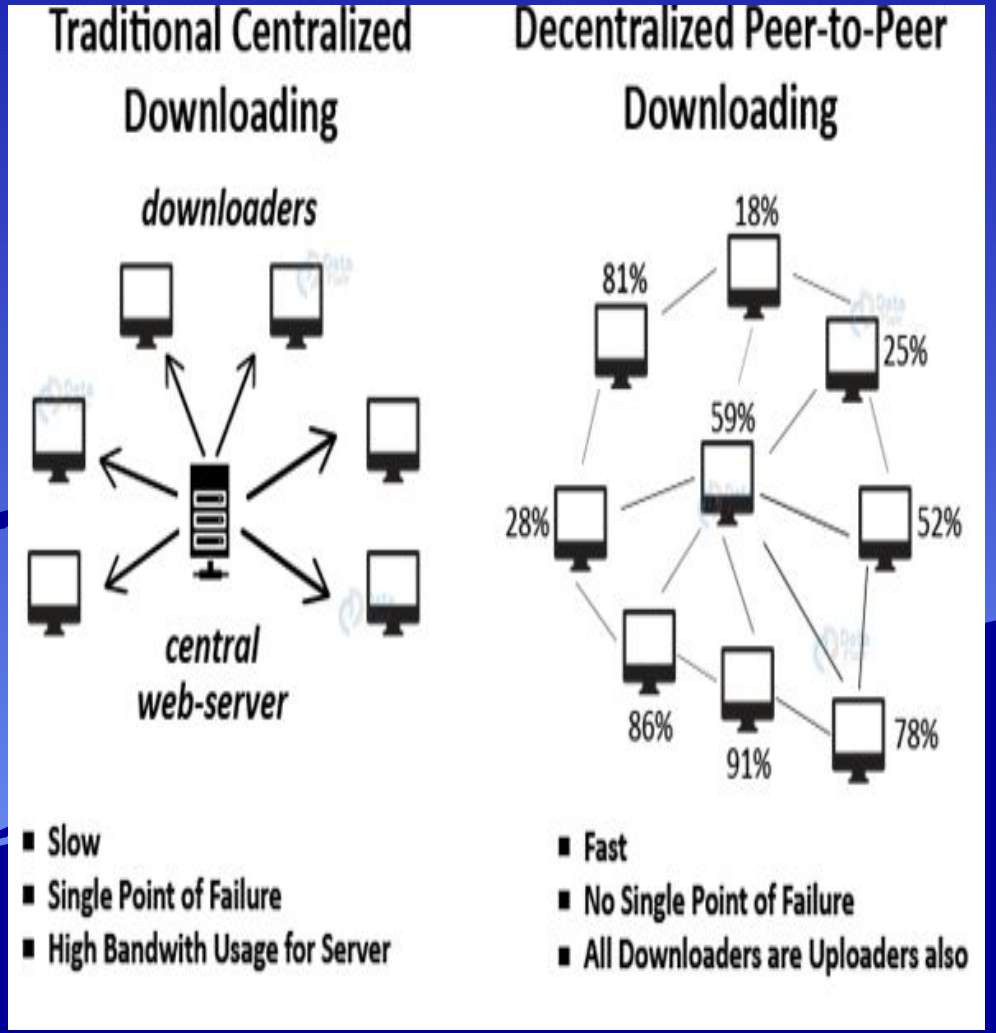
- Bitcoin launched in 2009.
- The Bitcoin blockchain became the first successful implementation of blockchain technology.



What is a blockchain

A blockchain is a constantly growing ledger which keeps a permanent record of all the transactions that have taken place in a secure, chronological, and immutable way.

- Ledger: It is a file that is constantly growing.
- Permanent: It means once the transaction goes inside a blockchain, you can put up it permanently in the ledger.
- Secure: Blockchain placed information in a secure way. It uses very advanced cryptography to make sure that the information is locked inside the blockchain.
- Chronological: Chronological means every transaction happens after the previous one.
- Immutable: It means as you build all the transaction onto the blockchain, this ledger can never be changed.



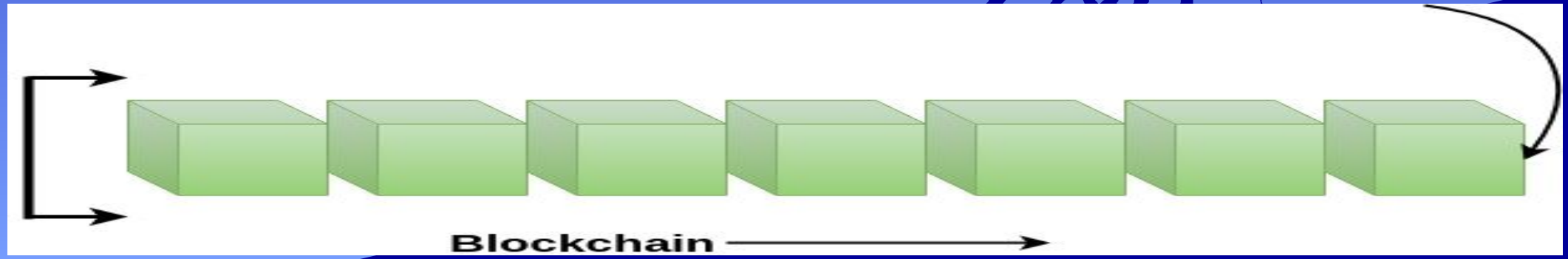
What is a blockchain

- A blockchain is “a distributed database that maintains a continuously growing list of ordered records, called blocks.” These blocks “are linked using cryptography. Each block contains a cryptographic hash of the previous block, a timestamp, and transaction data.
- Blockchain is a secure series or chain of timestamped records stored in a database that a group of users manages who are a part of a decentralized network.
- Blockchain is a decentralized or distributed ledger where each node in the network has access to the data or records stored in a blockchain.
- A blockchain is a chain of blocks which contain information. Each block records all of the recent transactions, and once completed goes into the blockchain as a permanent database. Each time a block gets completed, a new block is generated.

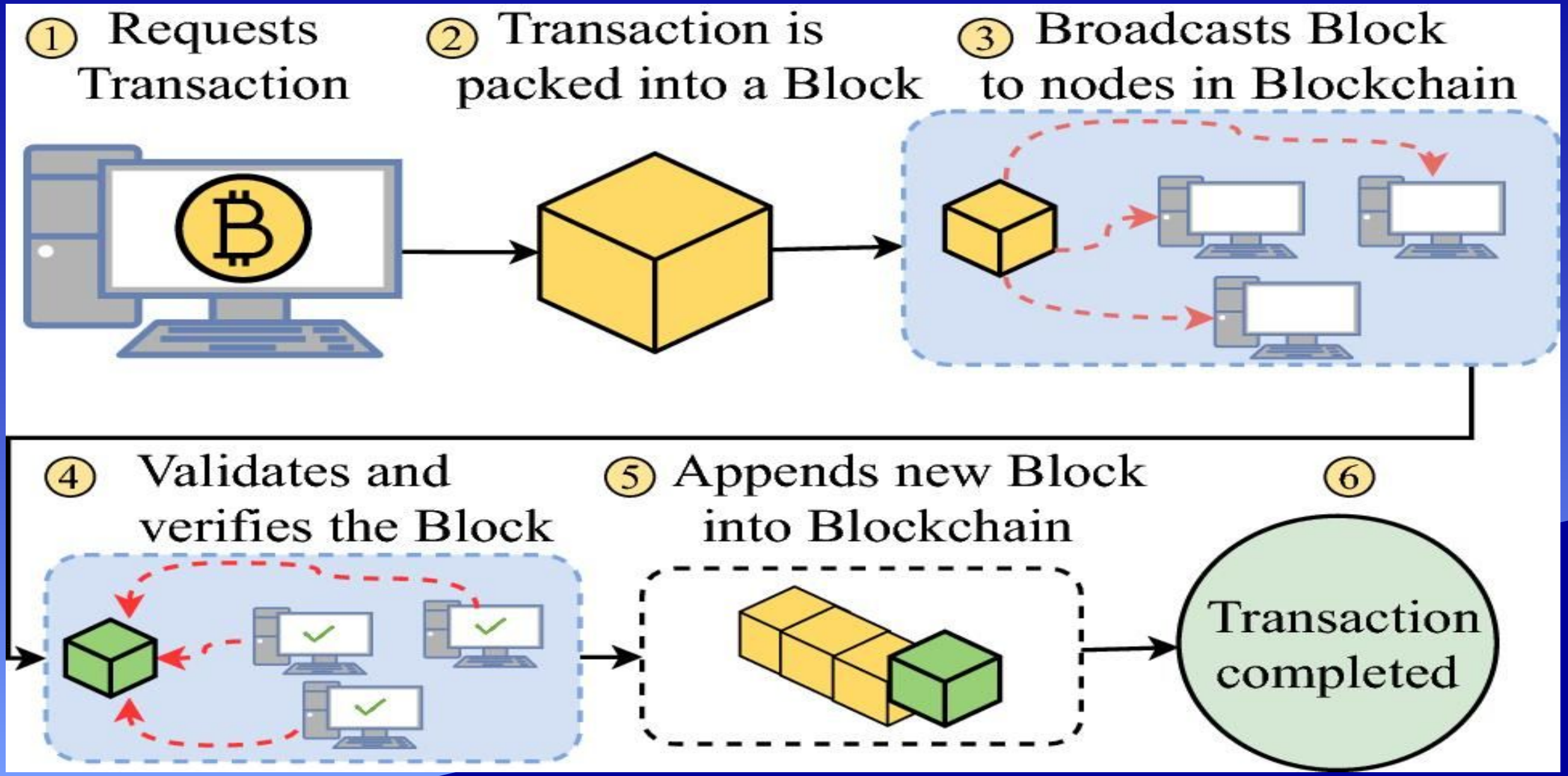
Who uses the blockchain

A blockchain can be used for the secure transfer of money, property, contracts, etc. without requiring a third-party intermediary like bank or government. Blockchain is a software protocol, but it could not be run without the Internet (like SMTP used in email).

Blockchain technology can be integrated into multiple areas. The primary use of blockchains is as a distributed ledger for cryptocurrencies. It shows great promise across a wide range of business applications like Banking, Finance, Government, Healthcare, Insurance, Media and Entertainment, Retail, etc.



How BlockChain Work



How Blockchain Works

1 **A transaction is requested** : A person wants to send or receive money/data.

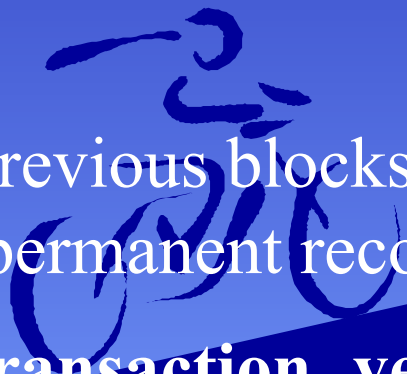
2 A block is created: The transaction information is put into a digital block.

3 **The block is shared:** The block is sent to many computers (called **nodes**) in the network.

4 **The transaction is verified:** The computers check whether the transaction is valid.

5 **The block is added to the blockchain**

- After verification, the block is connected to previous blocks.
- The transaction is completed and becomes a permanent record.



Blockchain works by creating a block for a transaction, verifying it through a network of computers, and then adding it permanently to a chain of blocks.

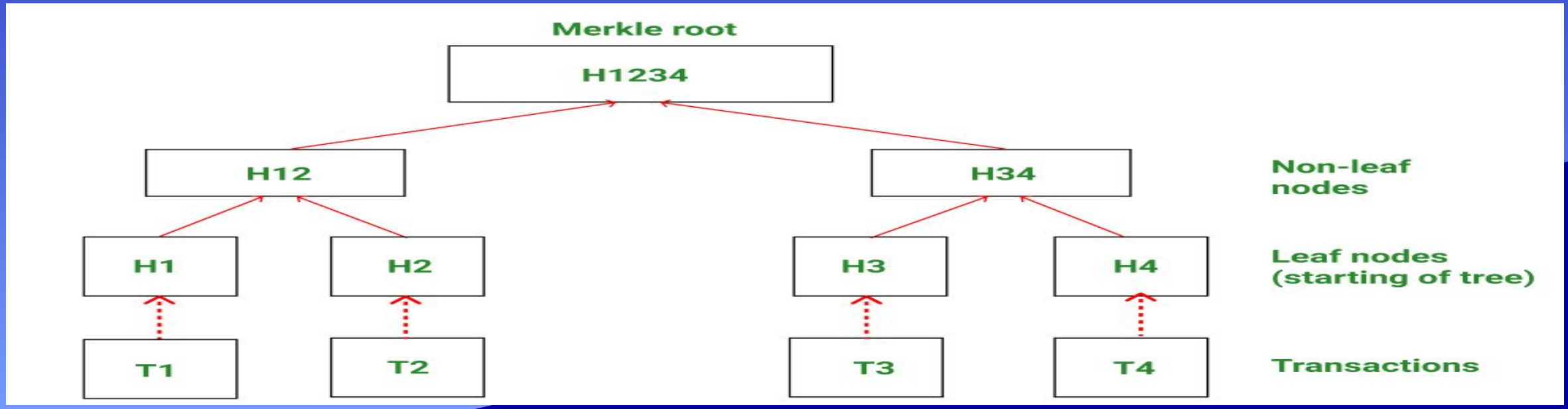
Merkle trees

What is a Merkle Tree: A Merkle Tree is a tree-like data structure used in blockchain to efficiently and securely verify data.

It combines the hashes of transactions until a single hash, called the Merkle Root, is obtained.

A Merkle Tree is a structure that organizes transaction hashes into a tree and produces one final hash (Merkle Root) to verify all transactions quickly.

A Merkle Tree is a tree structure used in blockchain to organize and verify transactions efficiently. Each transaction is hashed, pairs of hashes are combined and hashed again, and this process continues until a single hash called the Merkle Root is obtained. The Merkle Root represents all transactions in the block.



Merkle trees

1. Transaction Blocks

- T1, T2, T3, and T4 represent transaction data blocks.
- These are the original transactions stored in the blockchain.

2. Hash of Transactions : Each transaction is converted into a unique hash value:

  - Hash(T1)
  - Hash(T2)
  - Hash(T3)
  - Hash(T4)

3. Pairing of Hashes : Hashes are combined in pairs and hashed again:

  - Hash(T1, T2)
  - Hash(T3, T4)

4. Root Hash (Merkle Root) : The paired hashes are again combined and hashed to produce the final hash called the **Root Hash** or **Merkle Root**.

Merkle trees

Working of Merkle Tree

1. Transactions are converted into hashes.
2. Hashes are grouped in pairs.
3. Paired hashes are hashed again repeatedly.
4. The final single hash obtained is called the Merkle Root.

Advantages of Merkle Tree

- Fast verification of transactions
- Improved data security
- Efficient storage and data management
- Detects any change in transaction data

Applications

- Blockchain systems like Bitcoin and Ethereum
- Data integrity verification
- Distributed systems
- Peer-to-peer networks



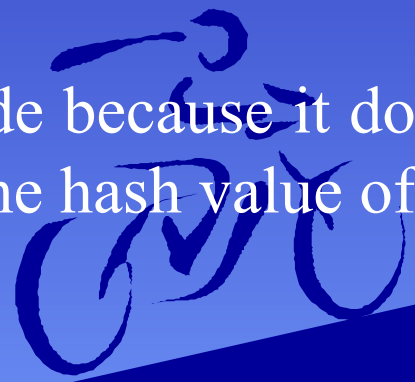
Components of blockchain

Following are the components of a Blockchain network –

- Node
- Ledger
- Wallet
- Nonce
- Hash

1. Node – It is of two types – Full Node and Partial Node.

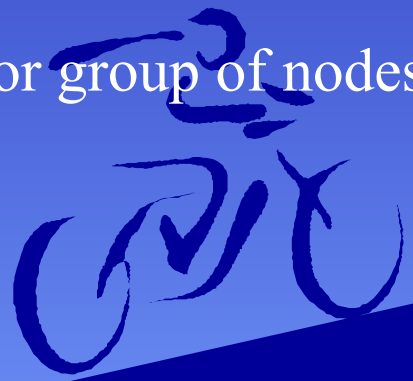
- Full Node – It maintains a full copy of all the transactions. It has the capacity to validate, accept and reject the transactions.
- Partial Node – It is also called a Lightweight Node because it doesn't maintain the whole copy of the blockchain ledger. It maintains only the hash value of the transaction.



Components of blockchain

2. Ledger – It is a digital database of information. Here, we have used the term 'digital' because the currency exchanged between different nodes is digital i.e cryptocurrency. All Transaction are store in ledger. can be be any form(text,jpg) There are three types of ledger. They are –

- **Public Ledger –** It is open and transparent to all. Anyone in the blockchain network can read or write something.
- **Distributed Ledger –** In this ledger, all nodes have a local copy of the database. Here, a group of nodes collectively execute the job i.e verify transactions, add blocks in the blockchain.
- **Decentralized Ledger –** In this ledger, no one node or group of nodes has a central control. Every node participates in the execution of the job.



Components of blockchain

3. Wallet –It is a digital wallet that allows user to store their cryptocurrency. Every node in the blockchain network has a Wallet. Privacy of a wallet in a blockchain network is maintained using public and private key pairs. In a wallet, there is no need for currency conversion as the currency in the wallet is universally acceptable. Cryptocurrency wallets are mainly of two types

1. **Hot Wallet –** These wallets are used for online day-to-day transactions connected to the internet. Hackers can attack this wallet as it is connected to the internet. Hot wallets are further classified into two types –

- **a. Online/ Web wallets** – These wallets run on the cloud platform. Examples – MyEtherWallet, MetaMask Wallet.
- **b. Software wallets –**It consists of desktop wallets and mobile wallets. Desktop wallets can be downloaded on a desktop and the user has full control of the wallet. An example of a desktop wallet is Electrum.
- **c. Mobile wallets** – They are designed to operate on smartphone devices. Example – mycelium.

Components of blockchain

2 . Cold Wallet – These wallets are not connected to the internet. It is very safe and hackers cannot attack it. These wallets are purchased by the user. Example – Paper wallet, hardware wallet.

- **a. Paper wallet –** They are offline wallets in which a piece of paper is used that contains the crypto address. The private key is printed in QR code format. QR code is scanned for cryptocurrency transactions.
- **b. Hardware wallet –** It is a physical electronic device that uses a random number generator that is associated with the wallet.

4. Nonce – A nonce is an abbreviation for "number only used once," which is a number added to a hashed or encrypted block in a blockchain. It is the 32-bit number generated randomly only one time that assists to create a new block or validate a transaction. It is used to make the transaction more secure.

- It is hard to select the number which can be used as the nonce. It requires a vital amount of trial-and-error. First, a miner guesses a nonce. Then, it appends the guessed nonce to the hash of the current header. After that, it hashes the value and compares this to the target hash. Now it checks that whether the resulting hash value meets the requirements or not. If all the conditions are met, it means that the miner has created an answer and is granted the block.
- **5. Hash –**The data is mapped to a fixed size using hashing. It plays a very important role in cryptography. In a blockchain network hash value of one transaction is the input of another transaction.

Types: Public, Private, and Consortium

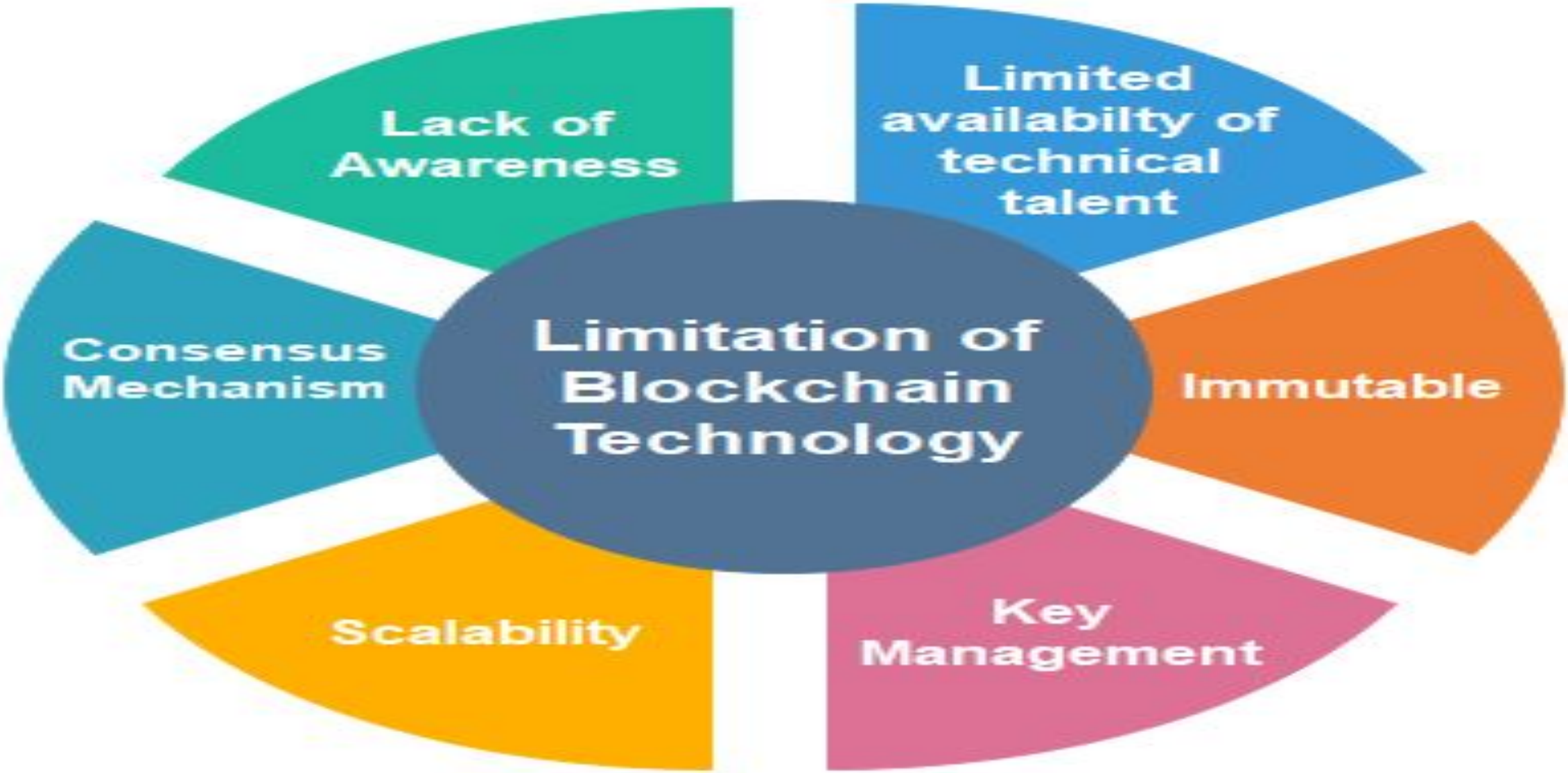
Property	Public Blockchain	Consortium Blockchain	Private Blockchain
Consensus determination	All miners	Selected set of nodes	One organization
Read permission	Public	May be public or restricted	May be public or restricted
Immutability	Almost completely tamper-proof	Potential for tampering	Potential for tampering
Efficiency	Low	High	High
Centralized	No	Partial	Yes
Consensus process	Permissionless	Permissioned	Permissioned

(Zheng et al., 2017)

Types: Public, Private, and Consortium

Feature	Public Blockchain	Private Blockchain	Consortium Blockchain
Access	Anyone can join	Only authorized users	Selected organizations can join
Control	No single owner	Controlled by one organization	Controlled by a group of organizations
Transparency	Fully transparent	Limited transparency	Partially transparent
Speed	Slower	Faster	Moderate
Security	Very high	High	High
Examples	Bitcoin, Ethereum	Company internal blockchain	Banking consortium blockchain

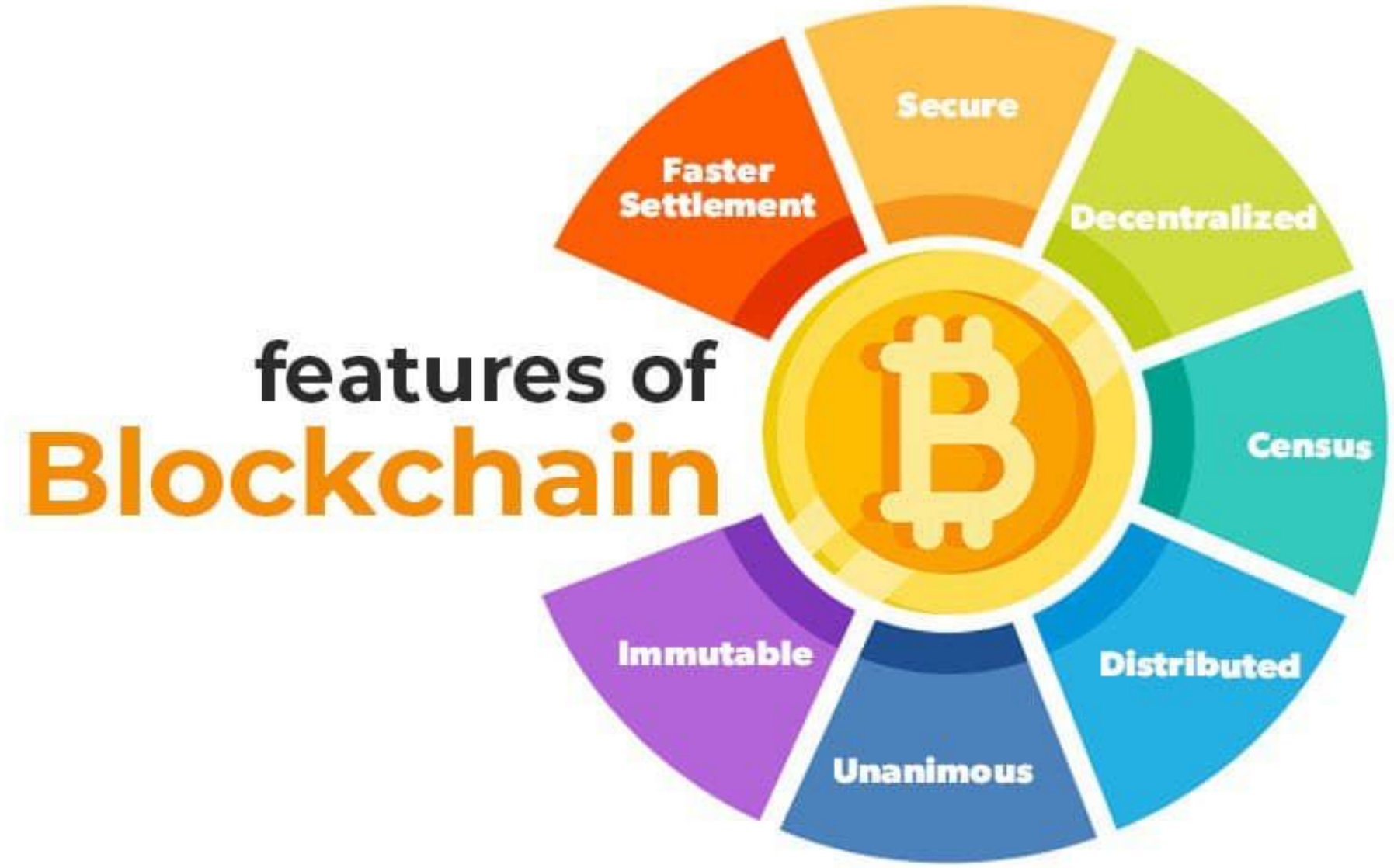
Limitations and Challenges of blockchain



Limitations and Challenges of blockchain

- **Lack of Awareness :** Many people do not understand how blockchain works.
- **Limited Availability of Technical Talent :** There are not enough blockchain experts and developers.
- **Immutability :** Once data is stored, it is very difficult to change or delete, even if there is a mistake.
- **Key Management :** If a user loses their private key (password), they may lose access to their data or cryptocurrency.
- **Scalability :** Blockchain can become slow when a large number of transactions occur at the same time.
- **Consensus Mechanism :** Verifying transactions requires agreement among network participants, which can take time and consume resources.

Features Of Blockchain



Features Of Blockchain

🔒 **Secure :** Blockchain uses cryptography to protect data from unauthorized access and tampering.

🌐 **Decentralized:**

- No single person or organization controls the blockchain.
- Control is shared among many computers.

🤝 **Consensus :** Transactions are verified only after network participants agree that they are valid.

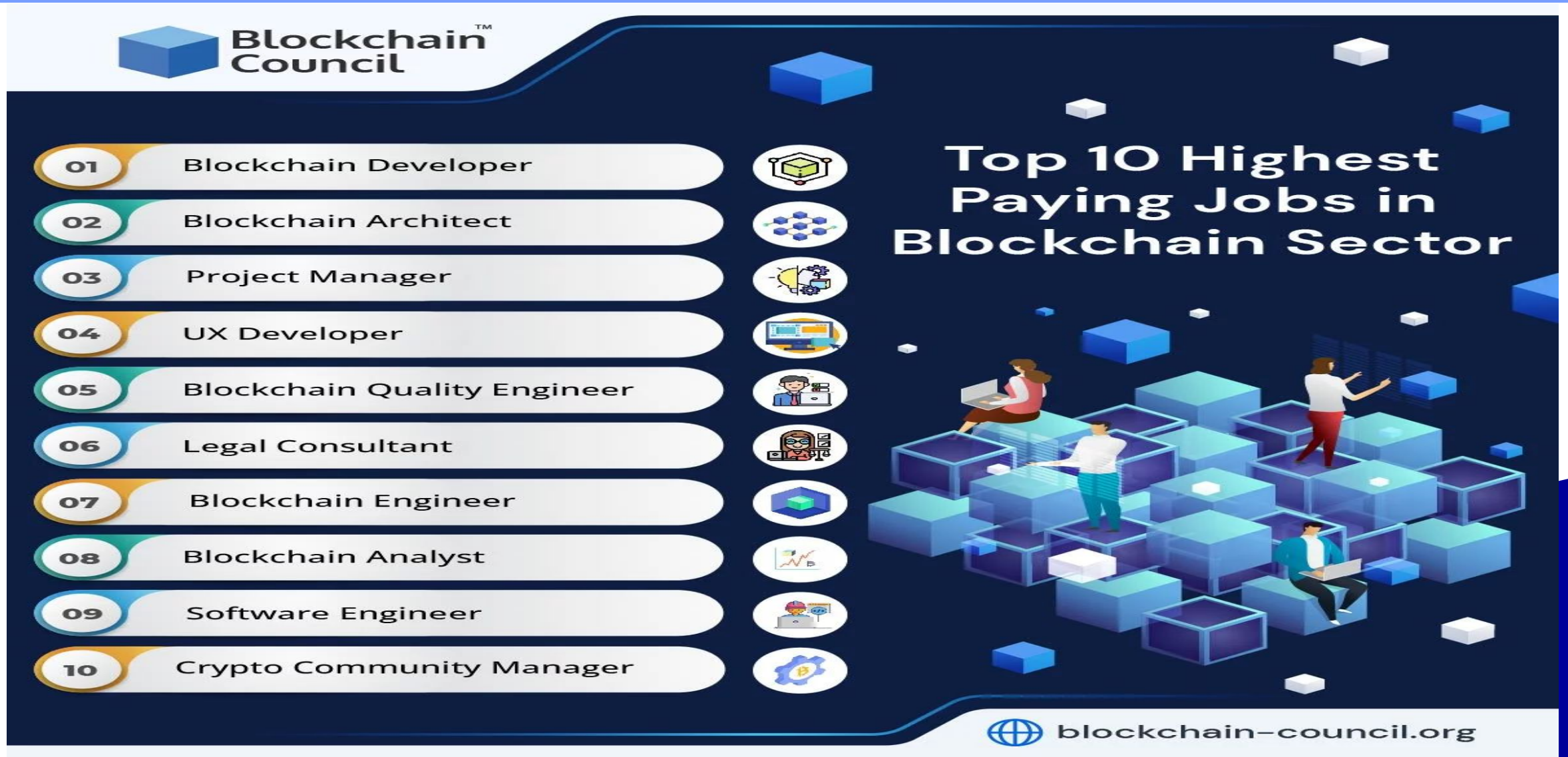
📡 **Distributed :** The same copy of the blockchain is stored on many computers (nodes).

✔ **Unanimous :** All participants see the same transaction history, ensuring consistency.

📝 **Immutable :**Once data is recorded, it cannot be easily changed or deleted.

⚡ **Faster Settlement :** Transactions can be processed and settled more quickly, especially without intermediaries.

Jobs in Blockchain Sector



THANK YOU



BlockChain



Mrs. Pallavi Patil
(Computer Engineering)

Module-2. Cryptocurrency

2.1 Cryptocurrency: Bitcoin, Altcoin, and Tokens (Utility and Security), Cryptocurrency wallets: Hot and cold wallets, Cryptocurrency usage, Transactions in Blockchain, UTXO and double spending problem

2.2 Bitcoin blockchain: Consensus in Bitcoin, Proof-of-Work (PoW), Proof-of-Burn (PoB), Proof-of-Stake (PoS), and Proof-of-Elapsed Time (PoET), Life of a miner, Mining difficulty, Mining pool and its methods



QBank

1. What is the fundamental difference between a hot wallet and a cold wallet in the context of blockchain and cryptocurrency storage?(5)
2. Differentiate between PoW,PoS,PoB & PoET.
3. What is mining difficulty and how is it calculated in a proof-of-work? Explain with an example.(10)
4. Mining pool and its methods.(5)
5. What is a double spending problem?How PoW solves the problem of double spending?(10)
6. How to calculate Mining difficulty in bitcoin.(5)
7. UTXO Model of Bitcoin.(5)
8. Explain Blockchain Transaction Life-cycle in details?



What is cryptocurrency

What is Cryptocurrency?

- The term cryptocurrency originated from the words crypto and currency.
- crypto means obscured or hidden and currency means money.

Cryptocurrency is a digital or virtual currency that uses cryptography for security and operates on blockchain technology.

Cryptocurrency is digital money that can be used to buy, sell, or transfer value online without a bank.

Key Features

- 💻 Digital – Exists only in electronic form.
- 🔒 Secure – Protected by cryptography.
- 🌐 Decentralized – Not controlled by any government or bank.
- ⚡ Fast Transactions – Can be sent anywhere in the world quickly.



Examples

- Bitcoin
- Ethereum

Cryptocurrency is a digital currency that uses blockchain technology and cryptography to secure transactions. It enables peer-to-peer transfers without the need for banks or other intermediaries.

What is cryptocurrency

- Cryptocurrencies are digital in nature that are designed to be faster, cheaper, and highly reliable.
- It is an open source encrypted leger of transaction built on blockchain technology.
- it is created via mining and users can transact directly with each other over the internet.
- Cryptocurrencies run on a distributed public ledger called blockchain, a record of all transactions updated and held by currency holders.
- Units of cryptocurrency are created through a process called mining, which involves using computer power to solve complicated mathematical problems that generate coins. Users can also buy the currencies from brokers, then store and spend them using cryptographic wallets.

Cryptocurrency: Bitcoin, Altcoin, and Tokens (Utility and Security)

There are a number of currencies in this world used for trading amenities. Rupee, Dollar, Pound Euro, and Yen are some of them. These are printed currencies and coins and you might be having one of these in your wallet.



1. What is Bitcoin?

Bitcoin is the world's first decentralized digital cryptocurrency, created in 2008 by Satoshi Nakamoto and launched in 2009.

Bitcoin is digital money that allows people to send and receive payments over the internet without using a bank.

Key Features

- 🔒 Secure – Uses cryptography for security.
- 🌐 Decentralized – No central authority controls it.
- ⚡ Peer-to-Peer – Transactions occur directly between users.
- 📖 Blockchain-Based – All transactions are recorded on a blockchain.

Bitcoin is a decentralized digital currency that operates on blockchain technology. It was created by Satoshi Nakamoto and allows secure peer-to-peer transactions without the involvement of banks or governments.



1. What is Bitcoin?

1.Bitcoin is the first blockchain that Satoshi Nakamoto designed as Bitcoin. it was designed as a decentralized, peer-to-peer network for sending money around the world outside traditional banking system.

2.The Bitcoin network is a peer-to-peer payment network. Therefore, it can have transactions only in the form of bitcoins. What it means is that we can send or receive bitcoins by moving them from one node in the network to another node in the network.

3.The most valuable cryptocurrency in the market. Considered by many as a form of “digital gold”, Bitcoin is popular as both an investment and a store of value. It’s accepted as a form of payment by many merchants and service providers worldwide.

4. But bitcoin is a currency you can not touch, you can not see but you can efficiently use it to trade amenities. It is an electronically stored currency. It can be stored in your mobiles, computers, or any storage media as a virtual currency.

What are Altcoins?

What are Altcoins?

Altcoins (Alternative Coins) are all cryptocurrencies other than Bitcoin.

Any cryptocurrency that is not Bitcoin is called an Altcoin.

Why Were Altcoins Created?

- To improve Bitcoin's features.
- To provide faster transactions.
- To support additional functions such as smart contracts and decentralized applications.

Examples of Altcoins

- Ethereum
- Litecoin
- Ripple
- Cardano



Altcoins are cryptocurrencies other than Bitcoin. They were developed to offer improvements or additional features such as faster transactions, lower fees, and smart contract functionality.

Tokens

- They are cryptocurrencies that have their own native blockchain. Ex. Ethereum,Ripple and Cardono. Tokens will operate on top of their native blockchain to facilitate the creation of decentralized applications.
- **Tokens are broadly classified into two types:**
- Utility tokens:
- Security tokens:



Tokens (Utility and Security Tokens)

Tokens (Utility and Security Tokens)

A **token** is a digital asset created on an existing blockchain.

1. Utility Token

A utility token gives access to a product, service, or feature within a blockchain platform.

Uses

- Access services
- Pay transaction fees
- Use applications on a blockchain network

Example

- Ethereum network tokens used for services and applications.

2. Security Token

A security token represents ownership or investment in an asset, such as a company, property, or shares.

Uses

- Represents ownership
- Can provide dividends or profit sharing
- Regulated by financial authorities

Example

- Digital shares of a company
- Tokenized real estate assets

Tokens (Utility and Security Tokens)

Feature	Utility Token	Security Token
Purpose	Access services	Represents ownership/investment
Value	Based on platform usage	Based on underlying asset
Regulation	Usually less regulated	Financially regulated
Example	App access token	Digital share token

What is Blockchain Cryptocurrency wallets ?

A **blockchain wallet** is a digital tool that allows you to **store, send, and receive cryptocurrencies** such as Bitcoin and Ethereum.

Despite the name, a blockchain wallet does **not actually store your coins**. The coins remain on the blockchain. The wallet stores your **cryptographic keys**:

- **Public key (or wallet address):** Like a bank account number that you can share with others to receive funds.
- **Private key:** Like a secret password that proves ownership and allows you to spend or transfer your cryptocurrency. Never share this key with anyone.



What is Blockchain Cryptocurrency wallets?

- A blockchain wallet is a software that enables sending and receiving cryptocurrencies such as Bitcoin, Ethereum, etc. It stores the record of transactions and also public and private keys which are used to perform transactions.
- A public key is similar to an account number. If A wants to send some money to B using Bitcoin when A sends the public key address to B. Anyone can send BitCoin using the public key.
- A private key is similar to an account password. Only the account holder knows the private key. The private key is used to send money.
- Public-Private keys are always present in pairs. A traditional wallet stores the currency. While a blockchain wallet never stores any cryptocurrencies. It contains the record of transactions performed by the users. It also stores the public and private keys of the user.

Cryptocurrency wallets

A blockchain wallet is a cryptocurrency wallet that is used to manage cryptocurrencies like Bitcoin and Ethereum. It helps to exchange funds easily and the transactions are more secure as they are cryptographically signed. The privacy and the identity of users are maintained and it provides all the features that are necessary for secure and safe transfers and exchange of cryptocurrencies. .

Features Of Blockchain Wallet:Secure: Blockchain wallets are secure as they carry on the private key that needs to be secured. There are no cryptocurrencies in blockchain wallets.

Easy to Use: Blockchain wallets are easy to use as they are just like any other digital wallets like Paytm that everyone uses in day-to-day life.

Low transaction fees: The cost of transferring funds using blockchain wallets is much lower than that of traditional bank transactions.

Instant transactions: Blockchain wallets allow users to instantly transfer funds across different geographics and these are barrier-free and without intermediaries.

Allows multiple cryptocurrencies: Blockchain wallets allow transactions across multiple cryptocurrencies and thus helping users to do easy currency conversions.



Cryptocurrency Wallet Types

Types of Blockchain Wallets

1. Hot Wallets (Connected to the Internet)

- Mobile apps
- Desktop software
- Web wallets

Examples:

- Trust Wallet
- MetaMask

Pros: Easy to use, convenient for daily transactions.

Cons: More vulnerable to hacking because they are online.

2. Cold Wallets (Offline Storage)

- Hardware wallets
- Paper wallets

Examples:

- Ledger Nano X
- Trezor Model T

Pros: Very secure.

Cons: Less convenient for frequent transactions.

Hot Wallets

Hot Wallets (Connected to the Internet)

A **hot wallet** is a type of blockchain wallet that is **connected to the internet**. It is used to store and manage cryptocurrencies for quick and easy transactions.

It is designed for online everyday transactions.it is always connected to the internet and hence it becomes a strong candidate for hackers. they are free,easy to set up, and convenient to use.Examples of hot wallets include exchange wallets, web wallets, and software wallets like Exodus

Features of Hot Wallets

- Always connected to the internet
- Easy to access using mobile or computer
- Fast sending and receiving of cryptocurrency
- Mostly free to use

Types of Hot Wallets

1. **Mobile Wallets** – Apps on smartphones
2. **Desktop Wallets** – Software installed on computers
3. **Web Wallets** – Wallets accessed through browsers

Examples

- Trust Wallet
- MetaMask
- Coinbase Wallet

Advantages

- Convenient and user-friendly
- Good for daily transactions
- Quick access to crypto assets
- Easy to connect with blockchain applications

Disadvantages

- More vulnerable to hacking and malware
- Requires internet connection
- Less secure than cold wallets

Cold Wallets

Cold Wallets (Offline Storage)

A cold wallet is a type of blockchain wallet that is not connected to the internet. It is mainly used for the safe storage of cryptocurrency.

It is a digital wallet this is not connected to the internet. they are not free. As they are available offline,they are more secure and used for long term storage of cryptocurrencies.

Features of Cold Wallets

- Works offline
- Provides high security
- Used for long-term storage of cryptocurrency
- Protects funds from online hacking

Types of Cold Wallets

1. **Hardware Wallets** – Physical devices used to store private keys
2. **Paper Wallets** – Printed copy of wallet keys or QR codes

Examples

- Ledger Nano X
- Trezor Model T

Advantages

- Very secure against cyber attacks
- Best for storing large amounts of cryptocurrency
- Private keys remain offline

Disadvantages

- Less convenient for daily transactions
- Physical device can be lost or damaged
- Usually costs money to buy hardware wallets

Different between Hot wallets and Cold wallets

Feature	Hot Wallets	Cold Wallets
Connectivity	Internet required	No internet, offline use
Security	Less secure, vulnerable to online threats	Most secure, immune to online hacks
Usage	Convenient for active trading and payments	Best for long-term storage
Cost	Most are free	May require initial purchase (e.g., hardware wallets)
DeFi Access	Direct access to DeFi platforms	Limited or no direct DeFi platform access
Private Key Control	User maintains private key	User maintains private key

Different between Hot wallets and Cold wallets

Feature	Hot Wallet	Cold Wallet
Internet Connection	Connected to the internet	Not connected to the internet
Security	Less secure	More secure
Speed of Transactions	Fast and convenient	Slower
Usage	Daily transactions	Long-term storage
Risk of Hacking	Higher	Very low
Cost	Usually free	Hardware wallets may cost money
Accessibility	Easy to access anytime	Less convenient
Examples	MetaMask, Trust Wallet	Ledger Nano X, Trezor Model T

Cryptocurrency Usage

Cryptocurrency is a digital currency used for different online financial activities. Some important uses are:

1. Online Payments: Cryptocurrencies like Bitcoin can be used to buy goods and services online.

2. Money Transfer: People can send money quickly across countries without banks.

3. Investment: Many people buy cryptocurrencies as an investment hoping their value increases in the future.

4. Trading: Cryptocurrencies are traded on crypto exchanges to earn profit from price changes.

5. Smart Contracts: Platforms like Ethereum support smart contracts, which automatically execute agreements without middlemen.

6. Decentralized Applications (DApps): Cryptocurrencies are used in blockchain-based applications such as games and finance apps.

7. Fundraising: Companies use crypto tokens for raising funds through ICOs (Initial Coin Offerings).

8. Digital Ownership: Cryptocurrency technology is used in NFTs (Non-Fungible Tokens) for ownership of digital art, music, and collectibles.



HOW BLOCKCHAIN WORKS

1



A transaction is requested

2



A Block that represents the transaction is created

3



The Block is sent to every node in the network

4



Nodes validate the transaction



Nodes receive a reward for the Proof of Work

5



The Block is added to the existing Blockchain

6



The transaction is complete

Blockchain Transaction Life-cycle

Step 1: Transaction is Requested

A person wants to send cryptocurrency (like Bitcoin) to another person.

Example: Alice sends Bitcoin to Bob.

Step 2: A Block is Created

The transaction information is placed into a digital block.

The block contains:

- Sender details
- Receiver details
- Transaction amount

Step 3: Block is Sent to the Network

The block is shared with many computers (called **nodes**) in the blockchain network.

Step 4: Nodes Validate the Transaction

The nodes check:

- Is the transaction valid?
- Does the sender have enough balance?
- Is the digital signature correct?

If everything is correct, the transaction gets approved.

Step 5: Block is Added to the Blockchain

After verification, the new block is connected to previous blocks, forming a chain called the **blockchain**.

Miners or validators may receive rewards for this process.

Step 6: Transaction is Complete

The transaction is successfully recorded permanently on the blockchain.

The receiver gets the cryptocurrency.

UTXO (Unspent Transaction Output)

UTXO stands for **Unspent Transaction Output**.
It is a method used in blockchain systems like Bitcoin to track cryptocurrency ownership.

Simple Meaning

A UTXO is the amount of cryptocurrency that is **received but not yet spent**.

You can think of it like **digital change or coins in your wallet**.

Simple Example

Suppose:

- Alice receives **10 BTC**
- This 10 BTC becomes a **UTXO**

Now Alice sends **6 BTC** to Bob:

- The full 10 BTC UTXO is used
- 6 BTC goes to Bob
- Remaining 4 BTC comes back to Alice as a new UTXO

So:

- Old UTXO is spent
- New UTXOs are created

How UTXO Works

1. User receives cryptocurrency
2. Received amount becomes a UTXO
3. When user spends crypto, old UTXO is consumed
4. New UTXOs are generated for receiver and change amount

Important Features

- Prevents double spending
- Increases transparency
- Every transaction can be tracked
- Used mainly in Bitcoin blockchain

UTXO Model

The **UTXO Model** is a transaction model used in blockchain systems like Bitcoin to manage and track cryptocurrency transactions.

In this model, cryptocurrency is represented as **unspent outputs** from previous transactions.

Simple Working of UTXO Model

Step 1: Receive Cryptocurrency

Suppose Alice receives **10 BTC**.

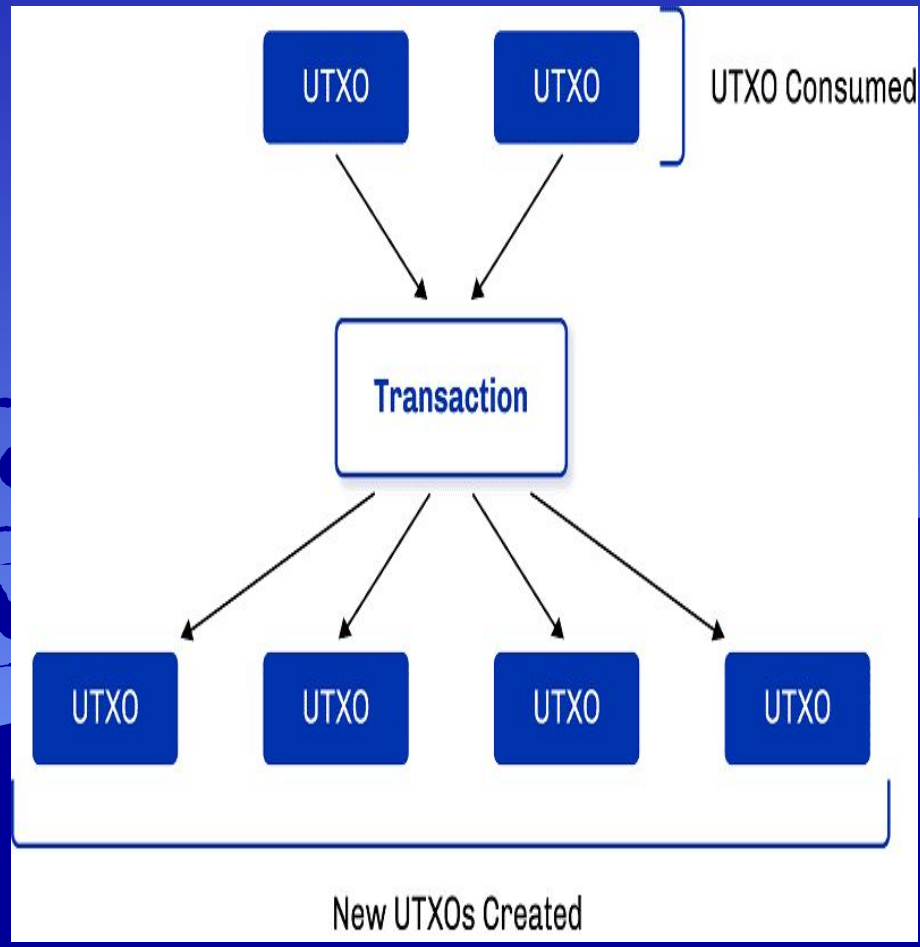
This 10 BTC becomes a **UTXO** (unspent transaction output).

Step 2: Spend Cryptocurrency

Alice wants to send **6 BTC** to Bob.

The system:

- Uses the entire 10 BTC UTXO
- Sends 6 BTC to Bob
- Returns remaining 4 BTC back to Alice as a new UTXO



Blockchain Double Spending

Double spending is a problem in digital currency where the same cryptocurrency is spent more than one time.

In simple words, a person tries to use the same digital coin for two different transactions.

- Double spending means spending the same money twice. As we know, any transaction can be processed only in two ways. One is offline, and another is online.
- Offline: A transaction which involves physical currency or cash is known as an offline transaction.
- Online: A transaction which involves digital cash is known as an online transaction.

Example



Suppose Alice has **1 Bitcoin**.

- Send the same 1 BTC to Bob
- And also send the same 1 BTC to Charlie

This is called **double spending** because the same money is used twice.

Why Double Spending Happens

Digital files can be copied easily. Without blockchain security, someone could copy digital currency and spend it multiple times.

How Bitcoin handles the Double Spending Problem?

1. Transaction Verification

When a user sends Bitcoin:

- The transaction is broadcast to the network.
- Nodes check:
  - Does the sender have enough balance?
  - Has this Bitcoin already been spent?

If the coin was already used, the transaction is rejected.

2. UTXO Model

Bitcoin uses the **UTXO (Unspent Transaction Output) model**.

- Every Bitcoin transaction creates outputs.
- Unspent outputs can be used in future transactions.
- Once a UTXO is spent, it cannot be reused.

This prevents the same Bitcoin from being spent twice.

3. Proof of Work (PoW)

Bitcoin miners solve complex mathematical problems.

The first miner to solve the problem:

- Validates transactions
- Creates a new block
- Adds it to the blockchain

This process secures the network and prevents fake transactions.

4. Blockchain Confirmations

After a transaction is added to a block:

- More blocks are added after it
- Each new block gives additional confirmation

More confirmations make it almost impossible to reverse or duplicate a transaction.

5. Decentralized Network

Bitcoin is controlled by thousands of nodes worldwide.

All nodes maintain the same blockchain copy and agree on valid transactions.

This decentralized verification prevents fraud.

What Is a Consensus Mechanism?

A **consensus mechanism** is a method used in blockchain networks to help all computers (**nodes**) agree on which transactions are valid.

It ensures that everyone in the network has the same copy of the blockchain.

Consensus means **agreement**.

In blockchain, thousands of computers must agree before adding a new block to the blockchain.

Why Consensus Mechanism is Important

- Prevent fraud and double spending
- Maintain blockchain security
- Validate transactions
- Keep all blockchain copies synchronized



How It Works

1. A transaction is created
2. Network nodes verify the transaction
3. Nodes follow consensus rules
4. Agreement is reached
5. Valid block is added to blockchain

Proof of Work (PoW)

Proof of Work (PoW) is a blockchain consensus mechanism used to validate transactions and add new blocks to the blockchain. **It is mainly used by Bitcoin.**

In PoW, computers called **miners** solve difficult mathematical problems.

The first miner to solve the problem:

- Verifies transactions
- Creates a new block
- Adds the block to the blockchain
- Receives cryptocurrency rewards

How Proof of Work Works

Step 1: Transaction is Created: Users make cryptocurrency transactions.

Step 2: Transactions are Collected: Miners collect many transactions into a block.

Step 3: Miners Solve Puzzle: Miners compete to solve a complex mathematical puzzle.Powerful computers ,High electricity

Step 3: Miners Solve Puzzle: Miners compete to solve a complex mathematical puzzle. Powerful computers, High electricity

Step 4: Block is Verified: When a miner solves the puzzle: Other nodes verify the solution,The block becomes valid

Step 5: Block Added to Blockchain: The verified block is added permanently to the blockchain. The miner receives rewards.

Advantages of PoW

- Very secure
- Prevents double spending
- Decentralized system
- Difficult to hack

Disadvantages of PoW

- Consumes large electricity
- Slow transaction processing
- Requires expensive hardware

Proof of Stake (PoS)

Proof of Stake (PoS) is a blockchain consensus mechanism used to validate transactions and add new blocks to the blockchain.

Instead of miners solving puzzles, PoS selects validators based on the amount of cryptocurrency they hold and lock in the network, called **staking**.

- Users lock some cryptocurrency as a stake
- The blockchain selects a validator
- The validator verifies transactions and creates a new block
- The validator receives rewards

How Proof of Stake Works

Step 1: Users Stake Cryptocurrency: Validators deposit or lock cryptocurrency into the network.

Step 2: Validator Selection: The system selects a validator based on: Amount of stake ,Sometimes staking duration Users with higher stake have a greater chance of selection.

Step 2: Validator Selection: The system selects a validator based on: Amount of stake,Sometimes staking duration.Users with higher stake have a greater chance of selection.

Step 3: Transaction Verification: Selected validator checks and verifies transactions.

Step 4: New Block Creation: Validator creates a new block and adds it to the blockchain.

Step 5: Rewards Given: Validator receives rewards for honest validation.

Advantages of PoS

- Uses less electricity
- Faster transactions
- More energy efficient
- No expensive mining hardware required

Disadvantages of PoS

- Users with more coins may get more power
- Can reduce decentralization in some cases

Proof of Burn (PoB)

Proof of Burn (PoB) is a blockchain consensus mechanism where users permanently destroy (burn) some of their cryptocurrency to gain the right to validate transactions and create new blocks.

- Users send coins to an unusable wallet address
- Those coins cannot be used again
- By burning coins, users show commitment to the network
- Users may receive mining rights or rewards

How Proof of Burn Works

Step 1: User Burns Coins

The user sends cryptocurrency to a special address called a **burn address**.

The coins are permanently destroyed.

Step 2: Network Verifies Burn

Blockchain nodes verify that the coins are actually burned.

Step 3: Mining/Validation Rights

The user gets a chance to validate transactions or create blocks.

Usually: More burned coins = Higher chance of selection

Step 4: Rewards

The validator receives rewards for adding valid blocks.

Advantages of PoB

- Less electricity consumption than Proof of Work
- Helps maintain network security
- Reduces coin supply, which may increase value

Disadvantages of PoB

- Coins are permanently lost
- Can favor wealthy users
- Less commonly used

Proof of Elapsed Time (PoET)

Proof of Elapsed Time (PoET) is a blockchain consensus mechanism where computers wait for a random amount of time, and the computer with the shortest waiting time gets the right to create the next block.

It is designed to use **less electricity** compared to Proof of Work (PoW).

- Each node receives a random waiting time
- Nodes wait for their assigned time
- The node whose timer finishes first creates the next block

How PoET Works

Step 1: Random Wait Time Assigned

Each validator/node gets a random waiting time from the system.

Step 2: Nodes Wait : All nodes stay idle and wait for their timer to finish.

Step 3: First Timer Ends: The node with the shortest waiting time wakes up first.

Step 4: Block Creation: That node validates transactions and creates a new block.

Step 5: Block Added to Blockchain

The new block is verified and added to the blockchain.

Advantages of PoET

- Low energy consumption
- Faster than Proof of Work
- Fair random selection process
- No need for expensive hardware

Disadvantages of PoET

- Mostly used in permissioned/private blockchains
- Requires trusted execution environments

Differentiate between PoW,PoS,PoB & PoET.

Feature	Proof of Work (PoW)	Proof of Stake (PoS)	Proof of Burn (PoB)	Proof of Elapsed Time (PoET)
Working Method	Miners solve mathematical puzzles	Validators are selected based on stake	Users burn coins to gain validation rights	Validators wait for a random time
Resource Used	Computing power and electricity	Staked cryptocurrency	Burned cryptocurrency	Waiting time
Energy Consumption	Very high	Low	Low	Very low
Block Creator	Miner	Validator	User who burned coins	Node with shortest waiting time
Hardware Requirement	Powerful hardware needed	Normal systems sufficient	No heavy hardware	No expensive hardware
Speed	Slower	Faster	Moderate	Fast
Security	Very high	High	High	High
Cost	Expensive	Less expensive	Coins are permanently lost	Low cost
Main Advantage	Strong security	Energy efficient	Reduces coin supply	Low power usage
Main Disadvantage	High electricity usage	V n concentration possible	Permanent coin loss	Mostly for private blockchains

Mining difficulty

Mining difficulty is a measure of how hard it is for miners to solve the mathematical puzzle required to add a new block to a blockchain.

It is mainly used in Bitcoin and other Proof of Work (PoW) blockchains.

Mining difficulty controls:

- How difficult mining is
- How fast new blocks are created

If many miners join the network: Difficulty increases

If miners leave the network: Difficulty decreases

Why Mining Difficulty is Important

- Maintain blockchain security
- Control block creation speed
- Prevent too many blocks from being created quickly

How Mining Difficulty Works

Step 1: Miners Solve Puzzle : Miners try to solve a complex mathematical problem.

Step 2: Difficulty Level Set: Blockchain sets a difficulty target.

- Higher difficulty = harder puzzle
- Lower difficulty = easier puzzle

Step 3: Block Creation Time Controlled: One block is created approximately every 10 minutes. The network automatically adjusts difficulty to maintain this timing.

Advantages of Mining Difficulty

- Keeps blockchain stable
- Increases security
- Prevents spam and fake blocks

Disadvantages

- High difficulty requires expensive hardware
- Consumes large electricity

What Is a Mining Pool?

A **mining pool** is a group of cryptocurrency miners who combine their computing power to mine blocks together.

When the pool successfully mines a block, the reward is shared among all members based on their contribution. Mining pools are mainly used in Proof of Work blockchains like Bitcoin.

Mining alone is very difficult because mining difficulty is high.

- Join together
- Share computing power
- Increase chances of earning rewards

How a Mining Pool Works

Step 1: Miners Join the Pool: Many miners connect their computers to a mining pool.

Step 2: Combined Computing Power: All miners work together to solve the mining puzzle.

Step 3: Block Successfully Mined

If the pool solves the puzzle first:

- A new block is added to the blockchain
- The pool receives the mining reward

Step 4: Reward Distribution

The reward is divided among miners according to:

- Their contributed computing power

Advantages of Mining Pools

- Higher chance of earning rewards
- Regular and stable income
- Easier for small miners to participate

Disadvantages of Mining Pools

- Rewards must be shared
- Some pools may become too powerful
- Pool operators may charge fees

Mining Pool Methods

Mining pools use different methods to distribute mining rewards among miners. These methods decide how miners are paid for their contribution.

1. Pay-Per-Share (PPS) :Miners receive a fixed payment for every valid share they submit. Payment is given even if the pool does not mine a block immediately.

Advantages

- Stable and predictable income

Disadvantages

- Pool operator takes more risk
- Higher pool fees

2. Proportional Method: Rewards are shared according to the number of shares submitted by miners. Payment happens only after the pool successfully mines a block.

Advantages : Simple method

Disadvantages: Income may vary

3. Pay-Per-Last-N-Shares (PPLNS) :Rewards depend on the last "N" shares submitted before a block is mined. Encourages long-term participation.

Advantages

- Prevents pool hopping
- Better for loyal miners

Disadvantages : Payments are less predictable

4. Score-Based Method :Recent shares receive higher scores. Miners who stay longer in the pool earn better rewards.

Advantages : Reduces unfair mining behavior

Disadvantages: Slightly complex calculation

5. Full Pay-Per-Share (FPPS) : This is an improved version of PPS. Miners receive:Block rewards , Transaction fees

Advantages : Higher earnings

Disadvantages ;Higher pool fees

THANK YOU



BlockChain



Mrs. Pallavi Patil
(Computer Engineering)

Module-3. Programming for Blockchain

3.1 Introduction to Smart Contracts, Types of Smart Contracts, Structure of a Smart Contract, Smart Contract Approaches, Limitations of Smart Contracts

3.2 Introduction to Programming: Solidity Programming – Basics, functions, Visibility and Activity Qualifiers, Address and Address Payable, Bytes and Enums, Arrays-Fixed and Dynamic Arrays, Special Arrays-Bytes and strings, Struct, Mapping, Inheritance, Error handling



3.3 Case Study – Voting Contract App, Preparing for smart contract development

QBank

1. Describe how solidity supports multiple inheritance with an Example.(5)
2. Write and elaborate a code in solidity to explain visibility and activity qualifiers.(10)
3. Explain view function and pure function in solidity with suitable examples.(10)
4. Fallback function in Solidity(5)
5. Explain fixed and dynamic arrays in solidity with suitable examples.(10)
6. Write a program in solidity to implement multilevel inheritance.(10)
7. Write a program in solidity to check whether a number is prime or not.(10)
8. what is smart contract? How Crowdfunding platforms can be managed using smart contracts?
9. what is smart contract? Explain Different types of smart contracts?

Introduction to Smart Contracts

A **smart contract** is a self-executing digital agreement stored on a blockchain. It automatically performs actions when predefined conditions are met, without needing a middleman such as a bank or lawyer.

Smart contracts are widely used on blockchain platforms like Ethereum.

A **smart contract** is a **self-executing digital contract** where the terms of the agreement or rules are written into **code** and stored on a **blockchain**. When specific **conditions are met**, the contract **automatically executes** the agreed-upon actions — without requiring a third party.

Why Use Smart Contracts on a Blockchain?



Blockchain is a **decentralized**, **transparent**, and **tamper-proof** ledger. By running smart contracts on a blockchain:

- No one can alter the agreement after deployment.
- Execution is guaranteed once conditions are satisfied.
- Everyone can verify the actions and data.

Introduction to Smart Contracts

- The term was coined by Nick Szabo in 1996 . Szabo claimed that smart contract can be realized with the help of a public ledger.
- Smart contracts work by following simple “if/when…then…” statements that are written into code on a blockchain. A network of computers executes the actions when predetermined conditions are met and verified.
- A Smart Contract (or crypto contract) is a computer program that directly and automatically controls the transfer of digital assets between the parties under certain conditions. A smart contract works in the same way as a traditional contract while also automatically enforcing the contract. Smart contracts are programs that execute exactly as they are set up(coded, programmed) by their creators. Just like a traditional contract is enforceable by law, smart contracts are enforceable by code.

Introduction to Smart Contracts

Features of Smart Contracts

- Automatic execution
- Transparent
- Secure
- Immutable (hard to change)
- Fast and reliable

Advantages

- Saves time
- Reduces cost
- Removes middlemen
- Increases trust and security

Disadvantages

- Difficult to modify after deployment
- Coding mistakes may cause issues
- Legal acceptance is still evolving

Applications of Smart Contracts

- Cryptocurrency transactions
- Online voting systems
- Supply chain management
- Insurance claims
- Banking and finance
- Decentralized Applications (DApps)

How Smart Contracts Work (Simplified Steps)

A smart contract is like a digital automatic agreement.

Example: “If payment is received, then deliver the product automatically.”

1. Contract Rules are Written

Developers write rules and conditions in programming code.

2. Contract Stored on Blockchain

The smart contract is deployed on the blockchain network.

3. Conditions are Checked

Blockchain nodes monitor whether the required conditions are fulfilled.

4. Automatic Execution

When conditions become true:

- The contract executes automatically
- No human intervention is needed



Key Characteristics of Smart Contracts

Characteristic	Explanation
Automation	Automatically triggers and performs actions when conditions are met
Transparency	Code and transactions are visible to all participants on the blockchain
Security	Cryptographically secured and resistant to tampering
Immutability	Cannot be changed once deployed
Trustless	No need for intermediaries or third parties



Real-World Applications

Domain	Use Case
Finance	Loans, insurance, decentralized exchanges
Supply Chain	Auto-payments upon delivery confirmation
Gaming	Asset ownership & transfer (NFTs)
Voting	Transparent, tamper-proof digital voting
Real Estate	Digital ownership transfer and records

Types of Smart Contracts

Smart contracts can be classified into different types based on their use and working style.

Smart contracts are self-executing programs stored on a blockchain that automatically enforce and execute the terms of an agreement when predefined conditions are met. Based on their functionality and use cases, smart contracts can be categorized into several types:

1. **Smart Legal Contracts**
2. **Decentralized Autonomous Organizations (DAOs)**
3. **Application Logic Contracts (ALCs)**



Smart Legal Contracts

A **Smart Legal Contract** is a type of smart contract that automatically executes legal agreements using blockchain technology.

It combines: Legal terms and conditions, Automatic execution through computer code

A smart legal contract works like a digital legal agreement.

When the required conditions are fulfilled: The contract automatically performs the action.

No middleman is needed.

How Smart Legal Contracts Work

Step 1: Legal Terms are Written

The agreement conditions are written in code.

Step 2: Contract Stored on Blockchain

The smart legal contract is deployed on the blockchain.

Step 3: Conditions are Verified

Blockchain checks whether the agreed conditions are satisfied.

Step 4: Automatic Execution

If conditions are true: The contract automatically executes the legal action.

Features of Smart Legal Contracts

- Automatic execution
- Transparent
- Secure
- Reduces paperwork
- Faster processing

Advantages

- Saves time and cost
- Reduces human errors
- No middleman required
- Improves trust and security

Disadvantages

- Difficult to modify once deployed
- Legal regulations are still developing
- Coding mistakes may cause problems

Decentralized Autonomous Organizations (DAOs)

A **Decentralized Autonomous Organization (DAO)** is an organization that is managed by smart contracts and blockchain technology instead of a central authority.

- Rules are written in smart contracts
- Members make decisions through voting
- No single person controls the organization

A DAO is like an online community or company where:

- Members work together
- Decisions are made collectively
- Blockchain automatically follows the rules

How DAOs Work

Step 1: Rules are Written in Smart Contracts: The organization's rules are programmed into smart contracts.

Step 2: Members Join the DAO: People become members by holding DAO tokens or cryptocurrency.

Step 3: Proposals are Created: Members suggest ideas or changes.

- Funding a project
- Changing rules

Step 4: Voting Process: Members vote on proposals.

Voting power may depend on: Number of tokens owned

Step 5: Automatic Execution: If the proposal gets enough votes:

- The smart contract automatically executes the decision.

Features of DAOs

- Decentralized management
- Transparent decision-making
- Community voting
- Automated operations
- No central authority

Advantages

- Greater transparency
- Democratic decision-making
- Reduced management costs
- Global participation

Disadvantages

- Slow decision-making sometimes
- Smart contract bugs can cause problems
- Legal regulations are unclear in many countries

Application Logic Contracts (ALCs)

Application Logic Contracts (ALCs) are smart contracts used to manage the logic and operations of blockchain-based applications.

They help decentralized applications (DApps) interact with the blockchain.

ALCs contain the business rules and functions of an application.

- What actions should happen
- When actions should happen
- How users interact with the blockchain

How ALCs Work

Step 1: Rules are Programmed: Developers write application rules into smart contract code.

Step 2: Contract Deployed on Blockchain: The ALC is stored on the blockchain network.

Step 3: User Interacts with Application : Users perform actions using a DApp. Ex. **Sending** cryptocurrency, Playing blockchain games,Voting online

Step 4: Contract Executes Logic: The ALC automatically processes the request according to predefined rules.

Features of ALCs

- Automates application operations
- Secure and transparent
- Removes middlemen
- Supports decentralized applications

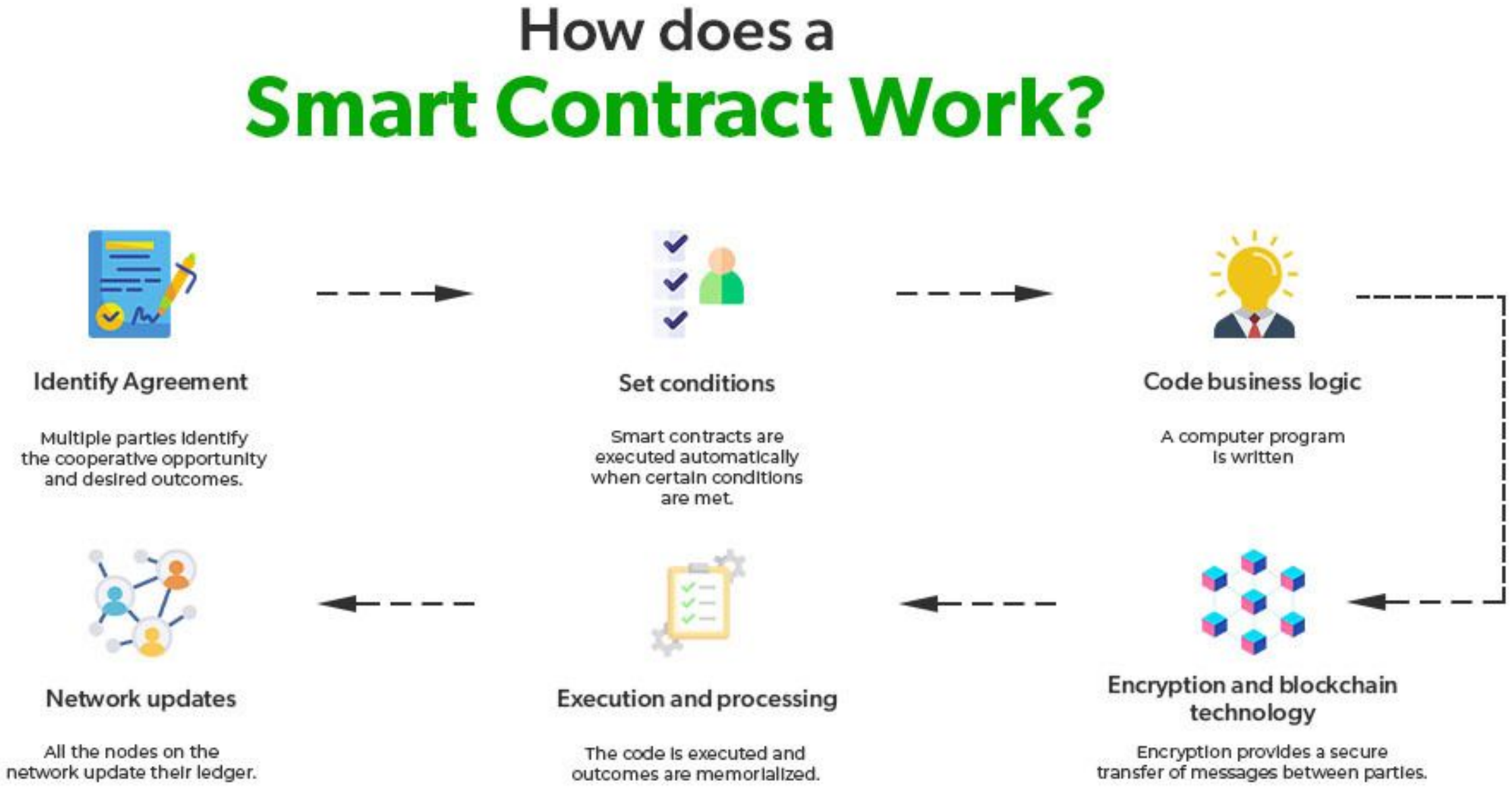
Advantages

- Fast processing
- Reliable automation
- Improves security
- Reduces manual work

Disadvantages

- Difficult to update after deployment
- Coding errors may affect applications
- Can become complex for large systems

Structure of a Smart Contract



Structure of a Smart Contract

1. Identify Agreement: Two or more people decide the agreement rules.

2. Set Conditions: The conditions and rules are written into smart contract code.

3. Code Business Logic: The smart contract stores all instructions and actions automatically.

4. Encryption and Blockchain Security: The contract is secured using blockchain encryption and stored on the blockchain network.

5. Execution and Processing : When the conditions are satisfied:

- The smart contract automatically executes the action.

Example:

- Payment received → Product delivered automatically.



6. Network Updates: All blockchain nodes update and verify the transaction, making it permanent and secure.

Smart Contract working

- Identify Agreement: Multiple parties identify the cooperative opportunity and desired outcomes and agreements could include business processes, asset swaps, etc.
- Set conditions: Smart contracts could be initiated by parties themselves or when certain conditions are met like financial market indices, events like GPS locations, etc.
- Code business logic: A computer program is written that will be executed automatically when the conditional parameters are met.
- Encryption and blockchain technology: Encryption provides secure authentication and transfer of messages between parties relating to smart contracts.
- Execution and processing: In blockchain iteration, whenever consensus is reached between the parties regarding authentication and verification then the code is executed and the outcomes are memorialized for compliance and verification.
- Network updates: After smart contracts are executed, all the nodes on the network update their ledger to reflect the new state. Once the record is posted and verified on the blockchain network, it cannot be modified, it is in append mode only.

Structure of a Smart Contract

1. SPDX License Identifier :// SPDX-License-Identifier: MIT

- Specifies the software license of the contract.
- Helps in legal and open-source identification.

2. Pragma Directive :pragma solidity ^0.8.0;

- Defines the Solidity compiler version.
- Ensures compatibility with a specific version.

3. Contract Declaration:contract MyContract

- Declares the smart contract.
- Similar to a class in object-oriented programming.

4. State Variables:uint public number;

- Variables stored permanently on the blockchain.
- Used to store contract data.

5. Constructor

```
constructor() {
  number = 0; }
```

- Executes only once when the contract is deployed.
- Used to initialize variables.

6. Functions:function setNumber(uint _num) public

- Contains the business logic of the contract.
- Can modify blockchain data.

7. View/Pure Functions:function getNumber() public view returns(uint)

- view → reads data without changing it.
- pure → does not read or modify blockchain data.

8. Access Modifiers :public,private,internal,external

- Control who can access variables and functions.

9. Events (Optional) :event NumberUpdated(uint newNumber);

- Used to log activities on the blockchain.
- Helps applications track changes.

10. Error Handling (Optional)

require()
assert()
revert()

- Used for validation and security checks.

Structure of a Smart Contract

A **smart contract** is a self-executing program deployed on a blockchain that enforces the terms of an agreement. It typically follows a defined structure to ensure proper functionality, security, and interaction.

1. Pragma Directive

Tells the compiler which Solidity version to use.

```
pragma solidity ^0.8.0;
```

2. Contract Declaration

Defines the smart contract with a name.

```
contract MyContract {
    // State variables, functions, events go here
}
```



Structure of a Smart Contract

3. State Variables

These are stored on the blockchain. They hold the contract's data.

```
string public name;
uint public balance;
address owner;
```

4. Constructor Function

Executed **once** when the contract is deployed. Used for initialization.

```
constructor() {
    owner = msg.sender; // the account that deploys the contract
}
```



Structure of a Smart Contract

5. Modifiers

Used to change the behavior of functions (e.g., access control).

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not the contract owner");
}
```

6. Functions

Functions define the logic of the smart contract. They can be:

- public, private, external, or internal
- view or pure (for non-state-changing)
- payable (for receiving Ether)



Limitations of Smart Contracts

Smart contracts are powerful, but they also have some limitations and challenges.

1. Immutability: Once a smart contract is deployed on the blockchain, it is very difficult to change or update it.

Problem:

- If there is a bug or mistake, it cannot be easily corrected.
- Errors may cause financial loss.

2. Security Risks: Smart contracts can contain coding vulnerabilities.

Problem:

- Hackers can exploit bugs.
- Poorly written contracts may lose funds.



3. Dependence on External Data : Smart contracts cannot directly access real-world information.

Problem:

- They need external services called **oracles**.
- Incorrect oracle data can affect contract execution

Limitations of Smart Contracts

4. Scalability Issues: Blockchain networks may become slow when many users use smart contracts simultaneously.

Problem:

- Transactions take more time.
- Network congestion occurs.

5. High Transaction Cost: Executing smart contracts requires **gas fees**.

Problem:

- Complex contracts consume more gas.
- Fees may become expensive during heavy traffic.

6. Legal and Regulatory Issues: Smart contracts are not fully recognized legally in every country.

Problem:

- Legal disputes may arise.
- Regulations are still evolving.



Limitations of Smart Contracts

7. Complexity in Development: Writing secure smart contracts requires advanced programming knowledge.

Problem:

- Small mistakes can create major issues.
- Testing and auditing are difficult.

8. Lack of Flexibility: Smart contracts execute automatically exactly as programmed.

Problem:

- They cannot easily handle human judgment or changing conditions.

9. Privacy Concerns: Blockchain data is usually public.

Problem:

- Contract data and transactions can be visible to everyone.



10. Irreversible Transactions: Transactions executed by smart contracts usually cannot be reversed.

Problem:

- Wrong transfers or errors cannot be easily corrected.

Smart Contract Approaches

Smart Contract Approaches refer to the different methodologies or models used to develop, deploy, and manage smart contracts on blockchain platforms. These approaches can vary based on use cases, platforms, automation level, and interaction models.

1. On-chain vs. Off-chain Smart Contracts

Approach	Description	Example
On-chain	Executes entirely on the blockchain. All logic and data are stored and processed on-chain.	Ethereum-based contracts (e.g., ERC-20 token)
Off-chain	Some logic or data is processed outside the blockchain (off-chain) and only critical parts are recorded on-chain.	Oracle-based contracts, payment channels

Smart Contract Approaches

2. Hybrid Smart Contracts

- Combines **on-chain code** with **off-chain components** (e.g., Chainlink oracles).
- Useful for contracts that require external data (like weather, stock prices, etc.).
- Enables smart contracts to interact with the real world.

3. Deterministic vs. Non-deterministic Approaches

Approach	Description
Deterministic	Same inputs always produce the same outputs. Essential for trustless execution on blockchains.
Non-deterministic	Relies on external data or events (through oracles), introducing variability. Used when real-world input is required.

Smart Contract Approaches

4. Manual vs. Automated Execution

Approach	Description
Manual Trigger	Execution starts through a user or system input (e.g., calling a function).
Automated	Runs automatically based on conditions (e.g., time-based smart contracts, DAOs, or event-triggered execution).

5. Contract-oriented Programming vs. Functional/Modular Approaches

- **Contract-oriented**: Monolithic contracts where everything is written in a single smart contract (common in Solidity).
- **Modular/Functional**: Breaks down logic into reusable modules or functions. Promotes scalability and upgradeability (e.g., proxy patterns).



What is Crowdfunding

Crowdfunding is a method of raising money from a large number of people through the internet to support a project, business, startup, social cause, or creative idea. Instead of getting money from one investor or bank, many people contribute small amounts of money. Crowdfunding is the process of collecting small amounts of money from many people online to fund a project or idea.

How Crowdfunding Works

1. A person or company creates a project idea.
2. The idea is posted on a crowdfunding platform.
3. People view the project online.
4. Interested people contribute money.
5. If enough funds are collected, the project starts.

Types of Crowdfunding

1. Donation-Based Crowdfunding: People donate money without expecting profit or reward.

Example: Charity and medical fundraising.



Types of Crowdfunding

2. Reward-Based Crowdfunding : Contributors receive rewards or products.

Example: Early access to a new gadget.

3. Equity-Based Crowdfunding: Investors receive shares or ownership in the company.

Example: Startup investment platforms.

4. Debt-Based Crowdfunding: Money is given as a loan and later repaid with interest.

Example: Peer-to-peer lending.

Advantages of Crowdfunding

- Easy way to raise funds
- Helps startups and small businesses
- Reaches many investors online
- Promotes innovative ideas
- Lowers dependency on banks

Disadvantages of Crowdfunding

- Risk of project failure
- Possible fraud or scams
- Competition among projects
- Platform fees may apply
- No guarantee of funding

Examples of Crowdfunding Platforms

- Kickstarter
- GoFundMe
- Indiegogo



How does BlockChain support Crowdfunding ?

Blockchain improves crowdfunding by making the fundraising process more secure, transparent, decentralized, and efficient.

Working of Blockchain in Crowdfunding

1. A project creator launches a crowdfunding campaign on a blockchain platform.
2. Investors send cryptocurrency or digital funds.
3. Smart contracts automatically manage rules and payments.
4. All transactions are stored on the blockchain.
5. Funds are released only when conditions are satisfied.

Ways Blockchain Supports Crowdfunding

1. Transparency: All transactions are recorded on the blockchain ledger.

Benefit

- Everyone can see how funds are collected and used.
- Reduces fraud and corruption.

2. Security: Blockchain uses cryptography to secure transactions.

Benefit

- Protects investor funds and project data.
- Difficult for hackers to alter records.

3. Decentralization: No central authority or bank is required.

Benefit

- Direct connection between investors and project creators.
- Reduces intermediary costs.

How does BlockChain support Crowdfunding ?

4. Smart Contracts: Smart contracts automatically execute fundraising rules.

Benefit

- Automatic release of funds
- No manual intervention needed
- Better trust between parties

Example

Funds are released only if the funding target is reached.

5. Global Participation: People from different countries can contribute using cryptocurrency.

Benefit

- Easier international fundraising
- Faster cross-border payments

6. Lower Transaction Costs: Blockchain reduces involvement of third parties.

Benefit

- Lower platform and banking fees
- Faster processing

7. Tokenization: Projects can issue digital tokens to investors.

Benefit

- Investors may receive rewards, ownership, or utility tokens.
- Easy transfer and trading of tokens.

Introduction to Programming: Solidity Programming

- Solidity is a brand-new programming language created by Ethereum which is the second-largest market of cryptocurrency by capitalization, released in the year 2015 and led by Christian Reitwiessner. Some key features of solidity are listed below:
- Solidity is a high-level programming language designed for implementing smart contracts.
- It is a statically typed object-oriented(contract-oriented) language.
- Solidity is highly influenced by Python, c++, and JavaScript which run on the Ethereum Virtual Machine(EVM).
- Solidity supports complex user-defined programming, libraries, and inheritance.
- Solidity is the primary language for blockchains running platforms.
- Solidity can be used to create contracts like voting, blind auctions, crowdfunding, multi-signature wallets, etc.

Solidity Programming

What is Solidity?

Solidity is a high-level programming language used to create **smart contracts** on blockchain platforms like Ethereum.

It is specially designed for developing decentralized applications (**DApps**) and smart contracts.

Solidity is an object-oriented, high-level programming language designed specifically for writing **smart contracts** on the **Ethereum blockchain** and compatible platforms (like BNB Chain, Polygon, Avalanche, etc.). Smart contracts are self-executing programs that run when certain conditions are met.

Solidity is statically typed, supports inheritance, libraries, and complex user-defined types, and is influenced by languages like JavaScript, Python, and C++.



Features of Solidity

- Object-oriented language
- Supports smart contract development
- Runs on Ethereum Virtual Machine (EVM)
- Supports inheritance, libraries, and user-defined types
- Secure and decentralized execution

Basic Concepts

1. Smart Contracts: A smart contract is a set of functions and data (state) that resides at a specific address on the Ethereum blockchain.

solidity

```
// A simple smart contract
pragma solidity ^0.8.0;

contract HelloWorld {
    string public message = "Hello, Ethereum!";
}
```



Basic Concepts

2. Variables & Data Types: Solidity supports several basic data types:

- uint, int – Unsigned/signed integers
- bool – Boolean
- address – Ethereum address
- string, bytes
- Arrays and structs

3. Functions

```
function getMessage() public view returns (string memory) {
    return message;
}
```



Basic Concepts

Functions can be:

- view – read-only, no gas used
- pure – returns result from parameters only
- payable – can receive Ether

4. State & Storage

- **Storage:** Persistent data on the blockchain
- **Memory:** Temporary data for function execution

5. Events : Used to log information on the blockchain.

```
event NewMessage(string message);
```

A Simple Example: Counter Contract

```
pragma solidity ^0.8.0;
contract Counter {
    uint public count = 0;
    function increment() public {
        count += 1;
    }
    function decrement() public {
        require(count > 0, "Counter is at zero");
        count -= 1;
    }
}
```



Solidity – Inheritance

What is Inheritance in Solidity?

- Inheritance is one of the most important features of the object-oriented programming language. It is a way of extending the functionality of a program, used to separate the code, reduces the dependency, and increases the re-usability of the existing code.
- Solidity supports inheritance between smart contracts, where multiple contracts can be inherited into a single contract. The contract from which other contracts inherit features is known as a base contract, while the contract which inherits the features is called a derived contract. Simply, they are referred to as parent-child contracts.
- The scope of inheritance in Solidity is limited to public and internal modifiers only

Inheritance is a feature in Solidity that allows one contract to acquire the properties and functions of another contract.



- Reuse code
- Reduce duplication
- Improve organization

The contract whose properties are inherited is called the **parent/base contract**, and the contract that inherits is called the **child/derived contract**.

Types of Inheritance in Solidity

1. Single Inheritance: In Single or single level inheritance the functions and variables of one base contract are inherited to only one derived contract.

One child inherits from one parent.

```
contract A {}
contract B is A {}
```

2. Multiple Inheritance: In Multiple Inheritance, a single contract can be inherited from many contracts. A parent contract can have more than one child while a child contract can have more than one parent.

One child inherits from multiple parents.

```
contract A {}
contract B {}
contract C is A, B {}
```

3. Multilevel Inheritance: It is very similar to single inheritance, but the difference is that it has levels of the relationship between the parent and the child. The child contract derived from a parent also acts as a parent for the contract which is derived from it.

Inheritance chain of multiple levels.

```
contract A {}
contract B is A {}
contract C is B {}
```

4. Hierarchical Inheritance: In Hierarchical inheritance, a parent contract has more than one child contracts. It is mostly used when a common functionality is to be used in different places

Multiple child contracts inherit from one parent.

```
contract A {}
contract B is A {}
contract C is A {}
```

Basic Syntax

```
// Base contract
contract A {
    uint public x;
    function setX(uint _x) public {
        x = _x;
    }
}
// Derived contract
contract B is A {
    function getX() public view returns (uint) {
        return x;
    }
}
```



Explain view function and pure function in solidity with suitable examples.

1. View Function: A view function reads the state but does not modify it. It cannot change variables stored on the blockchain. You use it when you just want to fetch or return data.

A **view function** can:

- Read blockchain data
- Access state variables
- But **cannot modify** data

Syntax

function functionName() public view returns(dataType)

Example of View Function

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ViewExample {

    uint public number = 10;

    function getNumber() public view
returns(uint) {
        return number;
    }
}
```



Explanation

- number is a state variable stored on blockchain.
- getNumber() only reads the value.
- It does not change any data.

Explain view function and pure function in solidity with suitable examples.

2. Pure Function: A pure function **neither reads Or writes** to the contract's state.
You use it for logic that is **independent** of the contract's storage, like calculations using only function arguments or local variables.

A **pure function**:

- Does not read blockchain data
- Does not modify blockchain data
- Works only with local variables or inputs

Syntax

function functionName() public pure returns(dataType)

Example of Pure Function

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PureExample {

   function addNumbers(uint a, uint b) public pure returns(uint) {
       return a + b;
   }
}
```



Explanation

- The function only uses input values a and b.
- No state variables are accessed.
- No blockchain data is changed.

Write and elaborate a code in solidity to explain visibility and activity qualifiers

In Solidity, **visibility qualifiers** control where variables and functions can be accessed, while **activity qualifiers** (view, pure, payable) define how functions interact with blockchain data and Ether.

1. Visibility Qualifiers

- public: Accessible from **anywhere** (inside, outside, derived contracts).
- private: Accessible **only inside** the contract that defines it.
- internal: Accessible **within the contract** and by **derived contracts**.
- external: Accessible **only from outside** the contract (not internally).

2. Activity Qualifiers

- view: Reads **state variables** but does **not change** them.
- pure: **Does not** read or modify state variables.
- payable: Allows the function to **receive Ether**.



Write and elaborate a code in solidity to explain visibility and activity qualifiers

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract QualifierDemo {
    // State Variable
    uint public number = 100;
    // PUBLIC Function
    function publicFunction() public pure returns(string memory)
{
        return "This is a Public Function";
    }
    // PRIVATE Function
    function privateFunction() private pure returns(string
memory) {
        return "This is a Private Function";
    }
    // INTERNAL Function
    function internalFunction() internal pure returns(string
memory) {
        return "This is an Internal Function";
    }
/ /EXTERNAL Function
    function externalFunction() external pure returns(string
memory) {
        return "This is an External Function";
    }
    // VIEW Function
    function getNumber() public view returns(uint) {
        return number;
    }
    // PURE Function
    function add(uint a, uint b) public pure returns(uint) {
        return a + b;
    }
    // PAYABLE Function
    function deposit() public payable {
    }
    // Function calling private function
    function callPrivate() public pure returns(string memory) {
        return "Private Function Called Internally";
    }
}
```

Fallback function in Solidity

A **fallback function** is a special function in Solidity that executes automatically when:

- A contract receives Ether without matching any function call.
- A non-existing function is called.

It is mainly used to:

- Receive Ether
- Handle unknown function calls
- Provide default behavior

Characteristics of Fallback Function

- Has no name
- Has no arguments
- Cannot return values
- Declared using fallback
- Usually marked external

Syntax

```
fallback() external payable {
   // code
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract FallbackExample {
    event Log(string message, address sender, uint amount);
    // Fallback Function
    fallback() external payable {
        emit Log("Fallback function called", msg.sender, msg.value);
    }
}
```

Explain fixed and dynamic arrays in solidity with suitable examples

Arrays in Solidity are used to store multiple values of the same data type.

There are two types of arrays:

1. Fixed Arrays
2. Dynamic Arrays

1. Fixed Array

A **fixed array** has a predefined size that cannot be changed after declaration.

Syntax

datatype[size] arrayName;

Example of Fixed Array

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract FixedArrayExample {
   uint[5] public numbers = [1, 2, 3, 4, 5];
}
```

Explanation

- uint[5]
  → Creates an array of size 5.
- Only 5 elements can be stored.
- Size cannot be increased or decreased.

Advantages

- Less gas consumption
- Faster access

Disadvantages

- Size cannot change

Explain fixed and dynamic arrays in solidity with suitable examples

2. Dynamic Array

A **dynamic array** does not have a fixed size.

Its size can increase or decrease during program execution.

Syntax

```
datatype[] arrayName;
```

Example of Dynamic Array

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract DynamicArrayExample {
   uint[] public numbers;
   function addNumber(uint num) public {
      numbers.push(num);
   }
   function getLength() public view returns(uint) {
      return numbers.length;
   }
}
```

Explanation

- uint[]
  → Dynamic array of unsigned integers.
- push()
  → Adds new elements to the array.
- length
  → Returns total number of elements.

Advantages

- Flexible size
- Easy to add/remove elements

Disadvantages

- Slightly higher gas cost

Write a program in solidity to check whether a number is prime or not

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract PrimeNumber {
    function checkPrime(uint num) public pure returns (string memory) {
        if (num <= 1) {
            return "Not Prime";
        }
        for (uint i = 2; i < num; i++) {

            if (num % i == 0) {
                return "Not Prime";
            }
        }
```

.



THANK YOU



BlockChain



Mrs. Pallavi Patil
(Computer Engineering)

Module 4. Public Blockchain

4.1	Introduction to Public Blockchain, Ethereum and its Components, Mining in Ethereum, Ethereum Virtual Machine (EVM), Transaction, Accounts, Architecture and Workflow, Comparison between Bitcoin and Ethereum
4.2	Types of test-networks used in Ethereum, Transferring Ethers using Metamask, Mist Wallet, Ethereum frameworks, Case study of Ganache for Ethereum blockchain. Exploring etherscan.io and ether block structure

QBank

1. What is a public blockchain? State some characteristics of public blockchain.
2. Describe the architecture of Ethereum.
3. Comparison of Bitcoin and Ethereum.
4. What is an EVM?
5. Explain Gas and Ethers in detail.
6. what are the types of test network used in ethereum blockchain?
7. What are the EVM Accounts & Explain?
8. Explain the Ethereum and its Components?



Introduction to Public Blockchain

A **Public Blockchain** is a type of blockchain network that is open to everyone.

Anyone can:

- Join the network
- Read data
- Send transactions
- Participate in validation and mining

No permission is required to access the blockchain.

A public blockchain is a decentralized blockchain network that anyone can access and use.



Features of Public Blockchain

1. Open Access: Anyone with internet access can join the network.

2. Decentralization: No single person or organization controls the blockchain.

3. Transparency: All transactions are visible to all participants.

4. Security: Uses cryptography and consensus algorithms for protection.

5. Immutability: Once data is added, it cannot be easily changed.

Working of Public Blockchain

1. A user creates a transaction.
2. The transaction is broadcast to the network.
3. Nodes verify the transaction.
4. Verified transactions are added to a block.
5. The block is linked to previous blocks.
6. The blockchain ledger updates on all nodes.

Examples of Public Blockchain

- Bitcoin
- Ethereum
- Litecoin

Public Blockchain

Advantages of Public Blockchain

Advantage	Explanation
Transparency	All records are publicly visible
Security	Difficult to hack or alter
Decentralization	No central authority
Trustless System	No need for intermediaries
Global Access	Anyone can participate

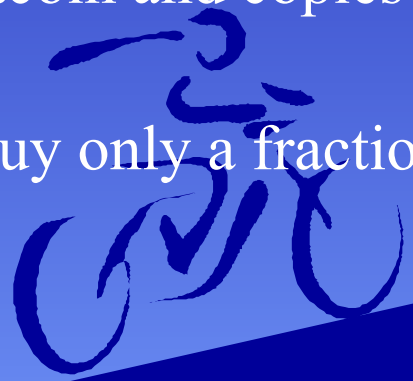
Disadvantages of Public Blockchain

Disadvantage	Explanation
Slow Transactions	Validation may take time
High Energy Consumption	Mining requires power
Scalability Issues	Performance decreases with heavy traffic
Less Privacy	Transaction details are public

Bitcoin

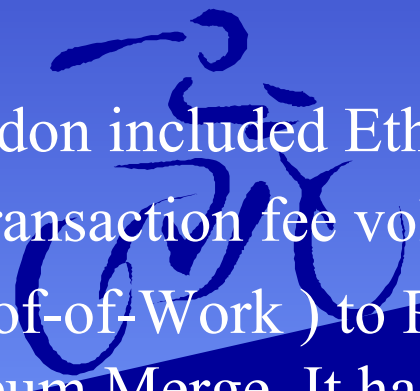
- Bitcoin is a digital currency that can be transferred on a peer-to-peer (P2P) network without the need for any central authority. It was invented by a person or group of people with the name Satoshi Nakamoto in 2008. All the transactions are stored in an immutable distributed ledger.

- Bitcoin is created, stored, transacted, and distributed using a decentralized, distributed system known as Blockchain.
- A public ledger records all the transactions of the Bitcoin and copies are retained on all the servers around the world.
- It is not necessary to buy an entire bitcoin, one can buy only a fraction of it if that is all necessary.
- BTC is the abbreviation for Bitcoin.



History of Ethereum

- 2013: Ethereum was first described in Vitalik Buterin's in 2013 with the goal of developing decentralized applications.
- 2014: In 2014, EVM was specified in a paper by Gavin Wood, and the formal development of the software also began.
- 2015: In 2015, Ethereum created its genesis block marking the official launch of the platform.
- 2018: In 2018, Ethereum took second place in Bitcoin in terms of market capitalization.
- 2021: In 2021, a major network upgrade named London included Ethereum improvement proposal and introduced a mechanism for reducing transaction fee volatility.
- 2022: In 2022, Ethereum has shifted from PoW(Proof-of-Work) to PoS(Proof-of-State) consensus mechanism, which is also known as Ethereum Merge. It has reduced Ethereum's energy consumption by ~ 99.95%.



What is Ethereum

- Ethereum is a decentralized blockchain platform that allows developers to build and run **smart contracts** and **decentralized applications (DApps)**.
- It is one of the most popular blockchain platforms after Bitcoin.
- Ethereum is a blockchain platform used for creating smart contracts and decentralized applications.
- Ethereum is a blockchain-based distributed platform. The network currency of Ethereum is known as Ether (ETH). Here also, the transactions are stored in an immutable distributed ledger.
- Ethereum is designed to be scalable, decentralized, and programmable.
- It provides a flexible platform to build applications using the solidity scripting language.
- Transactions are sent and received in user-created Ethereum accounts.
- It is a blockchain-based platform With the cryptocurrency Ether(ETH).
- Ethereum is like a decentralized computing network. It allows developers to create and run applications on its blockchain using smart contracts.
- Ethereum was initially released in 2015. Within two years of its release, it was ranked the second-best blockchain network, Bitcoin is the first. The Ethereum network acquired more global interest when China stated that it is the best blockchain network ever created.

Features of Ethereum

1. Decentralization: Ethereum is not controlled by any single authority or government.

2. Smart Contracts: Ethereum allows the creation and deployment of smart contracts. Smart contracts are created mainly using a programming language called solidity. Solidity is an Object Oriented Programming language that is comparatively easy to learn.

It allows automatic execution of digital agreements.

3. Decentralized Applications (DApps): Developers can create blockchain-based applications.

4. Cryptocurrency (Ether): Ethereum uses its own cryptocurrency called **Ether (ETH)**.

Ether is used for:

- Transactions
- Paying gas fees
- Running smart contracts



5. Ethereum Virtual Machine (EVM): It is designed to operate as a runtime environment for compiling and deploying Ethereum-based smart contracts.

The EVM executes smart contracts on the Ethereum network.

Feature	Bitcoin	Ethereum
Launch Year	2009	2015
Founder	Satoshi Nakamoto	Vitalik Buterin
Main Purpose	Digital currency	Smart contracts & DApps
Cryptocurrency	Bitcoin (BTC)	Ether (ETH)
Blockchain Type	Public blockchain	Public blockchain
Smart Contracts	Limited support	Strong support
Programming Language	Script-based	Solidity
Transaction Speed	Slower	Faster than Bitcoin
Consensus Mechanism	Proof of Work (PoW)	Proof of Stake (PoS)
Supply Limit	21 million BTC	No fixed maximum supply
Main Use	Peer-to-peer payments	Decentralized applications
Gas Fees	Transaction fees	Gas fees for contract execution
Token Creation	Limited	Easy token creation (ERC-20, NFTs)

What is Ethereum Virtual Machine and How it Works?

The **Ethereum Virtual Machine (EVM)** is the runtime environment of Ethereum that executes smart contracts.

It acts like a virtual computer that runs Solidity smart contract code on the Ethereum blockchain.

Ethereum Virtual Machine (EVM) is designed as the runtime environment for smart contracts in Ethereum. It is sandboxed and isolated from the other parts of the system. This means that any operation on EVM should not affect your data or programs in any way, no matter how many times you call a particular function on it.

EVM is a virtual machine that executes smart contracts on the Ethereum network.

Purpose of EVM

The EVM is responsible for:

- Executing smart contracts
- Processing transactions
- Updating blockchain state
- Ensuring network security and consistency

Features of EVM

1. Decentralized Execution

Every Ethereum node runs the same EVM.

2. Secure Environment

Smart contracts run in an isolated environment called a sandbox.

3. Turing Complete

EVM can perform complex computations and logic.

4. Deterministic

Same input always produces the same output on every node.

How Does EVM Works?

The **Ethereum Virtual Machine (EVM)** is the execution engine of Ethereum that processes and executes smart contracts on the Ethereum blockchain.

It works like a decentralized virtual computer shared by all Ethereum nodes.

1. Smart Contract Creation: Developers write smart contracts using Solidity.

Example:

```
contract Demo {
   uint public value = 10;
}
```

2. Compilation: The Solidity compiler converts the contract into:

- **Bytecode** → Machine-readable instructions
- **ABI (Application Binary Interface)** → Helps applications interact with the contract

3. Contract Deployment: The compiled bytecode is sent to the Ethereum blockchain through a transaction.

After deployment:

- The contract receives a unique blockchain address.
- The code becomes permanent on blockchain storage.

4. Transaction Request: When a user calls a smart contract function:

- A transaction request is sent to the network.
- Input data and gas fees are included.

5. EVM Execution: The EVM executes the bytecode instruction by instruction.

During execution, the EVM:

- Reads instructions
- Performs calculations
- Accesses storage and memory
- Updates blockchain state if needed

What are Ethereum Accounts?

In Ethereum, an **Ethereum account** is used to:

- Store Ether (ETH)
- Send and receive transactions
- Interact with smart contracts

Every account has:

- A unique address
- A balance
- Related transaction data

Ethereum accounts are digital accounts used to manage Ether and interact with the Ethereum

Types of Ethereum Accounts

Ethereum mainly has **two types of accounts:**

Type	Description
Externally Owned Account (EOA)	Controlled by users using private keys
Contract Account	Controlled by smart contract code

Externally CONTROLLED Accounts (EOAs)

In Ethereum, an **Externally Owned Account (EOA)** is an account controlled by a user through a **private key**.

It is the most common type of Ethereum account used for:

- Sending Ether (ETH)
- Receiving Ether
- Interacting with smart contracts

An Externally Owned Account (EOA) is a user-controlled Ethereum account managed using a private key.

Features of EOAs

1. Controlled by Private Key: Only the owner with the private key can access and control the account.

2. Can Send Transactions EOAs can:

- Transfer Ether
- Call smart contracts
- Start blockchain transactions

3. Stores Ether:EOAs hold cryptocurrency balances.

4. No Smart Contract Code:EOAs do not contain programmable code.

How EOAs Work

Step 1: Account Creation:A wallet generates:

- Public key
- Private key
- Ethereum address

Step 2: Transaction Creation: The user creates a transaction.

Example:

- Send ETH
- Interact with smart contract

Step 3: Digital Signature: The transaction is signed using the private key.

Step 4: Network Verification: Ethereum nodes verify the signature and transaction.

Step 5: Blockchain Update: After validation, the transaction is stored permanently.

Contract Accounts

In Ethereum, a **Contract Account** is an account controlled by **smart contract code** instead of a user. It is created automatically when a smart contract is deployed on the Ethereum blockchain. A Contract Account is an Ethereum account that contains smart contract code and executes automatically according to programmed rules.

Features of Contract Accounts

1. Controlled by Smart Contract Code: The account operates according to predefined program logic.

2. Created by Deployment:A contract account is created only when a smart contract is deployed.

3. Cannot Start Transactions by Itself : It cannot independently initiate transactions. It only responds when:

- A user
- Another contract

4. Stores Code and Data: Contract accounts contain:

- Smart contract code
- Storage variables
- Contract state data

How Contract Accounts Work

Step 1: Write Smart Contract:
Developers write code using Solidity.

Example:

```
contract Demo {
   uint public number = 10;
}
```

Step 2: Deploy Contract: The smart contract is deployed to Ethereum.

Step 3: Contract Address Creation:
Ethereum generates a unique contract address.

Step 4: Interaction: Users or applications call contract functions.

Step 5: Automatic Execution: The Ethereum Virtual Machine (EVM) executes the contract logic.

Ethereum – Gas and Fees

In Ethereum, Gas is the unit used to measure the computational work required to execute transactions and smart contracts.

Users must pay fees in Ether (ETH) to perform operations on the Ethereum network.

Gas is the cost required to execute transactions and smart contracts on Ethereum.

Ether (ETH) is the native cryptocurrency of Ethereum.

It is used to:

- Send and receive digital payments
- Pay transaction fees
- Execute smart contracts
- Reward validators

Ether is the digital currency used in the Ethereum blockchain network.

Uses of Ether

Use	Explanation
Payments	Send money between users
Gas Fees	Pay for computations
Smart Contracts	Execute decentralized applications
Investment	Cryptocurrency trading and holding

Units of Ether

Unit	Value
Wei	Smallest unit
Gwei	1 billion Wei
Ether	Main unit

Ethereum – Gas and Fees

2. Gas in Ethereum

What is Gas?

Gas is the unit used to measure computational work on Ethereum.

Every transaction and smart contract operation requires gas.

Gas is the fee required to perform operations on Ethereum.

Why Gas is Needed

Gas helps:

- Prevent spam attacks
- Avoid infinite loops
- Measure computation cost
- Reward validators

Gas Mechanism

Every Ethereum operation consumes a certain amount of gas.

Examples:

Operation	Gas Needed
ETH Transfer	Low
Smart Contract Deployment	High
Complex Calculations	More Gas

Ethereum – Gas and Fees

- Ethereum Gas is a section that calculates the quantity of calculation action that it takes to perform specific functions. Every function that carries position in Ethereum like transactions and smart contracts etc. performance needs some part of gas. It is essential to the blockchain P2P network because it is the power that authorizes it to accomplish exactly what an automobile needs fuel to drive. Gas refers to the cost required to complete a deal on the Ethereum network.
- **Facts about Ethereum Gas**
- Gas prices are delivered in the form of Ethereum's born money, the currency of Ethereum is ETH.
- Gas costs are indicated in "gwei" which is a movement of Ethereum, every single "gwei" is equivalent to 0.000000001 ETH.
- For sample, rather than stating that the gas costs 0.000000001 ether, say, the gas costs 1 "gwei".



GWEI

- Gwei is a combination of “giga” and “wei”. It is a sect of the blockchain technology ETH, this coin is operated on the Ethereum P2P network. ETH is a blockchain medium, like Bitcoin where users can make transactions with respect to buying and selling interests and benefits without the involvement of an intermediator.
- Example
- For example, Rahul needs to give 1 ETH to Shubham. During the transaction, the gas boundary is 21000 units, and the gas price is 200 gwei.
- Total fee costs = Gas units (limit) * Gas price per unit
- = 21000 * 200
- = 4,200,000 gwei (0.0042 ETH)



Components of Ethereum Network

- **Nodes –** There are two types of nodes in an Ethereum network. They are as follows.
- Mining Node – These nodes are responsible for writing all the transactions that have occurred in the Ethereum network in the block.
- Ethereum Virtual Machine Node – These are the nodes in the Ethereum network in which Smart Contracts are implemented. By default, this node utilizes a 30303 port number for the purpose of communication among themselves.
- **Ether –** Just like we have money to do the exchange of goods in real world, we need Ethers to exchange data or perform any kind of operation in Ethereum Blockchain. Ether is a type of cryptocurrency used in the Ethereum network just like a bitcoin is used in a blockchain network. It is a peer-to-peer currency, similar to Bitcoin. It tracks and promotes each transaction in the network.

Components of Ethereum Network

- **Gas** – Gas is an internal currency of the Ethereum network. We need gas to run applications on the Ethereum network, much as we need gas to run a vehicle.
- To complete every transaction on the Ethereum network, a consumer must first make a payment—send out ethers—and the intermediate monetary value is known as gas.
- Gas is a unit of measurement on the Ethereum network for the computing power used to execute a smart contract or a transaction.
- The price of gas is very low compared to Ether. The execution and resource utilization costs are predetermined in Ethereum in terms of Gas units, called gwei.
- **Ethereum Accounts –**
- There are two types of Ethereum accounts. They are as follows.
- **Externally** controlled **account (ECA) –**
- These accounts are used to store transactions.
- **Contract account –**
- As the name itself suggests, these accounts store the details of Smart Contracts.

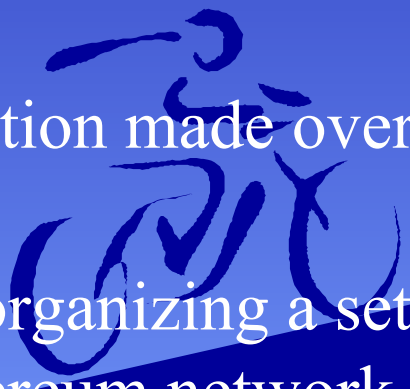


Components of Ethereum Network

- Nonce - For externally owned accounts, nonce means the number of transactions via this account. For a contract account, nonce means the number of contracts generated via this account.
- Storage Root –It is the main root node of a Merkle tree. Hash of all details of the account is stored here. The root of the Merkle tree is the verification of all transactions.
- Consensus Client –A consensus client is a software that implements the Proof-of-Stake Algorithm and helps the blockchain network ensure that all the nodes agree on the state of the blockchain.

It has the following key responsibilities

- Transaction Validation : Ensuring that any transaction made over the ethereum network adhere to the rules and guidelines set
- Block Proposal: Block proposal is the process of organizing a set of valid transaction into groups and then broadcasting them to the ethereum network for proposal as including them in the network.



Types of test-networks used in Ethereum

In Ethereum, **test networks (testnets)** are blockchain networks used for testing smart contracts and decentralized applications (DApps) without using real Ether.

Developers use testnets to:

- Test smart contracts
- Find bugs
- Practice transactions
- Experiment safely

Ethereum test networks are practice blockchain environments where developers test applications using free test Ether instead of real money.



Why Testnets are Important

- No financial risk

some of most popular testnets

Ropsten : was one of the oldest and most widely used Ethereum test networks.

It was designed to mimic the behavior of the Ethereum main network, allowing developers to test smart contracts and DApps without spending real Ether.

Ropsten was an Ethereum test network where developers could test applications using free test ETH instead of real ETH.

Features of Ropsten

1. Public Test Network: Anyone could join and use the network.

2. Similar to Ethereum Mainnet: Its behavior was very close to the real Ethereum network.

3. Free Test Ether: Developers could obtain test ETH from faucets.

4. Smart Contract Testing: Used for testing and debugging smart contracts before deployment.

How Ropsten Worked

1. Developer writes a smart contract.
2. Connects wallet to Ropsten.
3. Gets free test ETH.
4. Deploys the contract on Ropsten.
5. Tests transactions and contract behavior.

Advantages of Ropsten

- Free testing environment
- Similar to Ethereum mainnet
- Supported DApp development
- No financial risk

Limitations

- Network instability at times
- Spam attacks due to free participation
- Slower performance during heavy usage

some of most popular testnets

Kovan Testnet: Kovan was a popular Ethereum test network used by developers to test smart contracts and decentralized applications (DApps) without spending real Ether.

It used a **Proof of Authority (PoA)** consensus mechanism, making it faster and more efficient than some older testnets.

Kovan was an Ethereum testnet where developers could safely test blockchain applications using free test ETH.

Features of Kovan

1. Proof of Authority (PoA): Instead of mining, trusted validators approved transactions.

Benefits:

- Faster transactions
- Lower resource consumption
- Stable network performance



2. Free Test Ether: Developers could obtain test ETH from faucets.

Purpose:

- Deploy smart contracts
- Test DApps
- Perform transactions

some of most popular testnets

3. Public Test Network: Anyone could connect and use Kovan for development and testing.

4. Safe Testing Environment: No real money was involved.

How Kovan Worked

1. Developer writes a smart contract.
2. Connects a wallet (e.g., MetaMask) to Kovan.
3. Obtains free test ETH.
4. Deploys and tests the contract.
5. Fixes issues before deploying to the main Ethereum network.

Advantages of Kovan

- Fast transaction confirmation
- Free testing environment
- Stable network
- No financial risk



Limitations

- Test ETH had no real value
- Not identical to Ethereum mainnet behavior
- Eventually replaced by newer testnets

some of most popular testnets

Rinkeby was an Ethereum test network used by developers to test smart contracts and decentralized applications (DApps) without spending real Ether.

It used the **Proof of Authority (PoA)** consensus mechanism, where trusted validators confirmed transactions.

Rinkeby was a public Ethereum testnet that allowed developers to test blockchain applications using free test ETH.

Features of Rinkeby

1. Proof of Authority (PoA): Rinkeby used trusted validators instead of miners.

Benefits

- Faster transaction processing
- Low resource consumption
- Stable network performance

2. Free Test Ether: Developers could obtain free test ETH from faucets.

Uses: Deploy smart contracts,Test transactions,Develop DApp

3. Public Network: Anyone could connect and use the network for testing.

4. Safe Testing Environment: No real money was required, making it ideal for experimentation.

How Rinkeby Worked

1. Write a smart contract in Solidity.
2. Connect a wallet (e.g., MetaMask) to Rinkeby.
3. Obtain free test ETH.
4. Deploy and test the contract.
5. Fix errors before deploying to the Ethereum mainnet.

Advantages of Rinkeby

- Fast transaction confirmation
- Stable testing environment
- Free testing with test ETH
- Easy smart contract deployment

Limitations

- Test ETH had no real value
- Not exactly the same as Ethereum mainnet
- Limited validator participation

some of most popular testnets

Sokol :was a test network associated with the **POA Network (Proof of Authority Network)** and was commonly used for testing blockchain applications before deploying them to the POA main network.

It provided a fast, low-cost environment for developers to test smart contracts and decentralized applications (DApps).

Sokol is a testnet of the POA Network used for testing smart contracts and blockchain applications without using real cryptocurrency.

Features of Sokol

1. Proof of Authority (PoA): Sokol uses the **Proof of Authority** consensus mechanism.

Benefits

- Fast transaction processing
- Low transaction cost
- High efficiency

2. Free Test Tokens: Developers can obtain free test tokens for testing purposes.

3. Smart Contract Testing: Used to:Deploy smart contracts,Test DApps,Verify blockchain functionality

4. Public Access: Anyone can connect and use the network for development and testing.

How Sokol Works

1. Developer writes a smart contract.
2. Connects a wallet to the Sokol testnet.
3. Obtains free test tokens.
4. Deploys the contract.
5. Tests the application before moving to the main network.

Advantages of Sokol

- Fast confirmations
- Low-cost transactions
- Safe testing environment
- Easy smart contract deployment

Limitations

- Test tokens have no real value.
- Primarily intended for POA Network development.
- Less widely used than Ethereum testnet.

Connecting to a Testnet

Connecting to a **testnet** allows developers to test smart contracts and decentralized applications (DApps) on an Ethereum test network without using real Ether.

Connecting to a testnet means linking your wallet or development environment to an Ethereum testing network to perform transactions using free test ETH.

Requirements: Before connecting to a testnet, you need:

- An Ethereum wallet (e.g., MetaMask)
- Internet connection
- Testnet network details
- Test ETH from a faucet

Steps to Connect to a Testnet

Step 1: Install a Wallet: Install a wallet such as:

- MetaMask Create or import a wallet account.

Step 2: Open Network Settings: In MetaMask:

1. Click the network dropdown at the top.
2. Enable **"Show test networks"** in settings (if required).

Step 3: Select a Testnet: Choose an available testnet such as:

- Sepolia
- Holesky

Step 4: Obtain Test ETH: Get free test ETH from a testnet faucet.

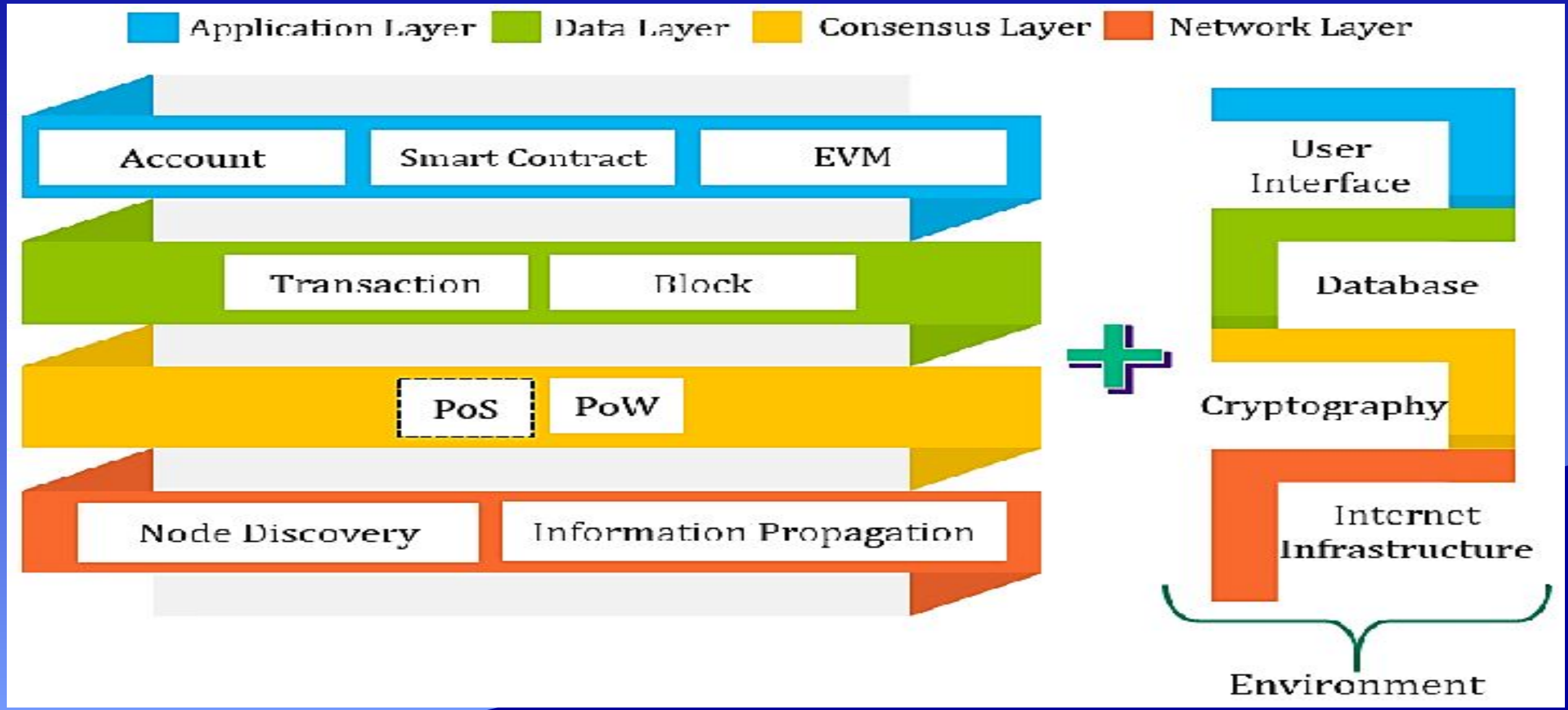
Test ETH:

- Has no real value
- Is used only for testing

Step 5: Verify Connection: After connecting:

- Your wallet will display the selected testnet.
- The balance will show test ETH.
- You can now deploy and test smart contracts.

Ethereum Architecture



Ethereum Architecture

Ethereum is a decentralized blockchain platform that enables the execution of **smart contracts** and **decentralized applications (DApps)**. Its architecture consists of several interconnected layers that ensure security, decentralization, and programmability.

The image shows the Ethereum Architecture, divided into four layers and an environment that supports the blockchain.

1. Application Layer (Blue Layer): This is the top layer where users and developers interact with Ethereum.

Components:

- Account: Stores ETH and interacts with the network.
- Smart Contract: Self-executing programs stored on blockchain.
- EVM (Ethereum Virtual Machine): Executes smart contract code.

Purpose:

- Runs DApps (Decentralized Applications)
- Executes smart contracts
- Manages user accounts



Ethereum Architecture

2. Data Layer (Green Layer): This layer stores blockchain data.

Components:

- Transaction: Records transfer of ETH or contract interactions.
- Block: Collection of verified transactions.

Purpose:

- Maintains transaction history
- Stores blockchain information permanently

3. Consensus Layer (Yellow Layer): This layer ensures agreement among all nodes.

Components:

- **PoW (Proof of Work)** – Used by Ethereum before 2022.
- **PoS (Proof of Stake)** – Current Ethereum consensus mechanism.

Purpose:

- Validates transactions
- Creates new blocks
- Maintains network security

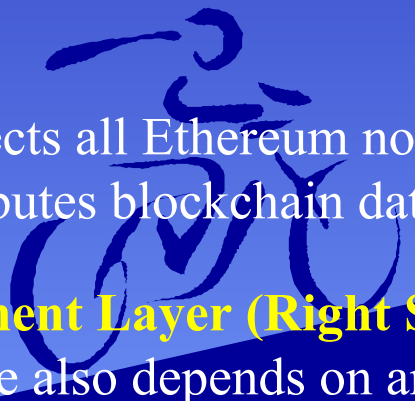
4. Network Layer (Orange Layer): Responsible for communication among Ethereum nodes.

Components:

- **Node Discovery**: Finds other nodes in the network.
- **Information Propagation**: Shares transactions and blocks among nodes.

Purpose:

- Connects all Ethereum nodes
- Distributes blockchain data



Environment Layer (Right Side): The Ethereum architecture also depends on an external environment.

Ethereum Architecture

User Interface: Allows users to interact with DApps and wallets.

Examples:

- MetaMask
- Web Applications

Database: Stores application-related data.

Cryptography

Provides:

- Security
- Digital signatures
- Hashing functions



Internet Infrastructure : **The** network through which Ethereum nodes communicate globally.

THANK YOU



BlockChain



Mrs. Pallavi Patil
(Computer Engineering)

Module 5. PRIVATE Blockchain

5.1	Introduction, Key characteristics, Need of Private Blockchain, Smart Contract in a Private Environment, State Machine Replication, Consensus Algorithms for Private Blockchain - PAXOS and RAFT, Byzantine Faults: Byzantine Fault Tolerant (BFT) and Practical BFT
5.2	Introduction to Hyperledger, Tools and Frameworks, Hyperledger Fabric, Comparison between Hyperledger Fabric & Other Technologies
5.3	Hyperledger Fabric Architecture, Components of Hyperledger Fabric: MSP, Chain Codes, Transaction Flow, Working of Hyperledger Fabric, Creating Hyperledger Network, Case Study of Supply Chain Management using Hyperledger

QBank

1. What is a private blockchain? State some characteristics of Private blockchain.
2. What is a private blockchain? Need of Private Blockchain.
3. Explain Smart Contract in a Private Environment.
4. Explain state machine replication with suitable examples.
5. Explain the RAFT consensus algorithm with a suitable example.
6. Explain the PAXOS Consensus algorithm for a private blockchain.
7. Explain Hyperledger Fabric V1 Architecture.
8. What is a backup in Practical Byzantine Fault Tolerance(PBFT)



Introduction to PRIVATE Blockchain

Private Blockchain is a type of blockchain network where access is restricted to selected users or organizations. Only authorized participants can join, read, write, or validate transactions in the network.

Unlike public blockchains, private blockchains are controlled by a single organization or a group of trusted members.

Definition

A private blockchain is a permissioned blockchain in which only approved participants can access the network and perform operations.

Features of Private Blockchain



1. Permission-Based Access: Only authorized users can participate.
2. High Security: Since participants are verified, the network is more secure and controlled.
3. Faster Transactions: Fewer nodes are involved, so transaction speed is high.
4. Better Privacy: Data is visible only to permitted members.
5. Energy Efficient:Does not require heavy mining like public blockchains.
6. Centralized Control: Managed by one organization or a consortium.

Introduction to PRIVATE Blockchain

Working of Private Blockchain

1. An organization creates the blockchain network.
2. Users are given permission to join.
3. Transactions are verified by selected nodes.
4. Approved transactions are added to the blockchain.
5. All authorized members can view updated records.

Advantages of Private Blockchain

- Faster processing speed
- More privacy and confidentiality
- Better scalability
- Lower transaction cost
- Suitable for businesses and enterprises

Disadvantages of Private Blockchain

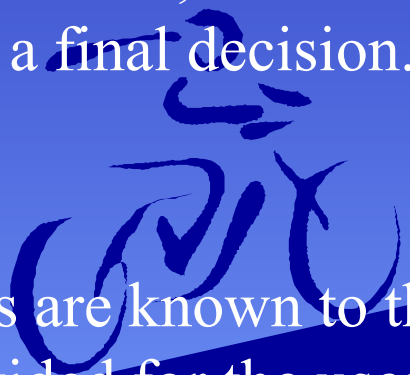
- Less decentralized
- Requires trust in the controlling organization
- Limited transparency
- Possibility of data manipulation by administrators

Applications of Private Blockchain

- Banking and Finance
- Supply Chain Management
- Healthcare Records
- Government Organizations
- Enterprise Data Management

PRIVATE Characteristics:

- **Permissioned**
- It is a closed blockchain, which means they are not accessible to the public or has a layer of access control. Only users with permission can access the network.
- **Privacy**
- Complete data privacy is their main focus. It is a more centralised blockchain and therefore authorisation is required before participating or accessing any data in the process.
- **High Performance**
- The number of participants in a private blockchain is limited, which is why fewer validators are required to carry out consensus protocols to reach a final decision. This makes its performance much faster and therefore easy to scale.
- **No Anonymity**
- In a private blockchain, there is no anonymity as users are known to the owner. Documentation and other information need to be provided for the user's identity to be confirmed.



Need of Private Blockchain

A **Private Blockchain** is needed when organizations want the benefits of blockchain technology while keeping data secure, private, and controlled.

Why Private Blockchain is Needed

1. Data Privacy and Security: Organizations handle sensitive information such as:

- Banking records
- Medical data
- Company documents

2. Faster Transaction Speed: Public blockchains are slower because many nodes verify transactions.
Private blockchains use limited trusted nodes, so transactions are processed quickly.

3. Controlled Access: Companies need control over:

- Who can join the network
- Who can read data
- Who can update records

Private blockchains provide permission-based access.

4. Lower Cost and Energy Usage: Private blockchains do not require heavy mining like public blockchains, reducing:

- Energy consumption
- Hardware cost
- Transaction fees

5. Better Scalability: Businesses process thousands of transactions daily. Private blockchains can handle more transactions efficiently.

6. Regulatory Compliance: Organizations must follow legal and government rules for:

- Data protection
- Financial reporting
- Audit tracking

Private blockchains help maintain secure and traceable records.

7. Improved Trust Within Organizations: Different departments or companies can securely share data without depending on a third party.

Smart Contract in a Private Environment

Working of Smart Contract in Private Blockchain

1. The organization creates a smart contract.
2. Rules and conditions are written in code.
3. Authorized users submit transactions.
4. The blockchain verifies the conditions.
5. If conditions are satisfied, the contract executes automatically.
6. The result is stored permanently on the blockchain.

Advantages

- Secure and trusted transactions
- Reduced paperwork
- Faster business operations
- Lower operational cost
- Better privacy and control
- No need for intermediaries

Disadvantages

- Less decentralization
- Requires maintenance by the organization
- Difficult to modify once deployed
- Dependence on trusted authorities

Applications

Banking: Automatic loan approval and payment processing.

Supply Chain: Tracking goods and verifying deliveries.

Healthcare: Secure sharing of patient records.

Insurance: Automatic claim settlement.

Real Estate: Digital property transfer agreements.

Smart Contract in a Private Environment

A **Smart Contract** in a private environment is a self-executing digital program that runs on a **private blockchain**.

It automatically performs actions when predefined conditions are met, but only authorized users can access or execute it.

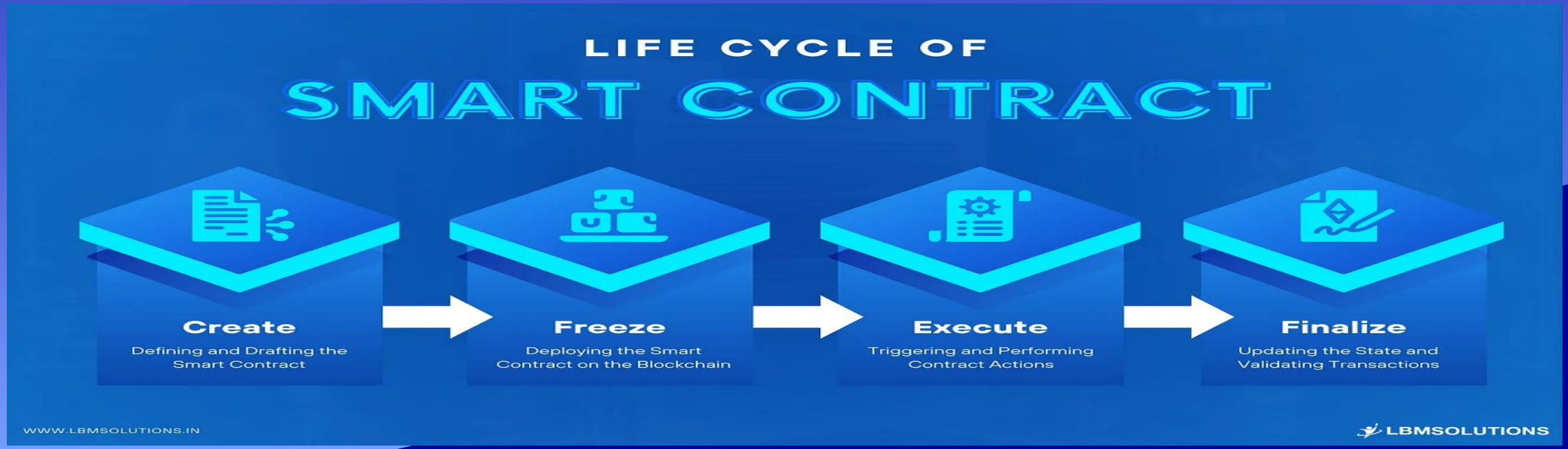
A smart contract in a private environment is an automated agreement stored on a private blockchain where access and execution are controlled by a trusted organization or group.

Features

1. **Permission-Based Access:** Only approved users or organizations can interact with the smart contract.
2. **High Security:** Transactions and contract data are protected within the private network.
3. **Automation:** The contract executes automatically without human intervention.
4. **Faster Execution:** Since fewer nodes participate, processing is quicker than in public blockchains.
5. **Privacy:** Sensitive business information remains confidential.

Smart Contract in a Private Environment

- Life Cycle of Smart Contract Development 📝
- Now that you understand the building blocks of a smart contract, let's explore the phases involved in its smart contract development and execution:
- 1 Create: Defining and Drafting the Smart Contract
- 2 Freeze: Deploying the Smart Contract on the Blockchain
- 3 Execute: Triggering and Performing Contract Actions
- 4 Finalize: Updating the State and Validating Transactions



Permissioned Blockchain – State Machine Replication

Introduction

A **Permissioned Blockchain** is a blockchain network where only authorized users can participate.
To keep all copies of the blockchain identical across different nodes, the system uses a concept called **State Machine Replication (SMR).**

What is State Machine Replication (SMR)?

State Machine Replication is a method used in distributed systems where multiple computers (nodes) maintain the **same data and execute the same operations in the same order**.

- Every node reaches the same final state.
- All copies of the blockchain remain synchronized.

Definition: State Machine Replication means:
"Replicating the same operations on multiple nodes so that all nodes produce the same output and maintain the same state."

Working of SMR in Permissioned Blockchain

Step 1: Client Sends Request

A user sends a transaction request.

Step 2: Ordering of Transactions

The network decides the correct order of transactions.

Step 3: Replication

The same ordered transaction is sent to all authorized nodes.



Step 4: Execution

Every node executes the transaction in the same manner.

Step 5: State Update

All nodes update their blockchain state identically.

Permissioned Blockchain – State Machine Replication

Components

1. State Machine: A system that changes from one state to another based on inputs. Example:

- Bank balance changes after a transaction.

2. Replication: Copying the same operations to multiple nodes.

3. Consensus Mechanism: Ensures all nodes agree on transaction order. Common algorithms:

- PBFT (Practical Byzantine Fault Tolerance)
- Raft
- Paxos

Advantages of SMR

- Data consistency
- High reliability
- Fault tolerance
- Faster recovery
- Secure transaction processing

Disadvantages

- Complex implementation
- Communication overhead
- Scalability challenges in large networks

Example: Suppose a bank uses a permissioned blockchain.

- A customer transfers ₹5000.
- The transaction is sent to all authorized bank servers.
- Every server updates the account balance identically.
- All servers now store the same updated balance.

This synchronization is achieved using State Machine Replication.

Applications

- Banking systems
- Supply chain management
- Healthcare networks
- Enterprise blockchain platforms

Example of State Machine: Smart Contract – Crowd-Funding

Introduction: A **Crowdfunding Smart Contract** is a blockchain-based application where people contribute money to support a project or business idea.

In blockchain, this system can be represented as a **State Machine**, where the smart contract changes from one state to another based on conditions and events.

State Machine: A **State Machine** is a model in which a system moves through different states depending on inputs or actions.

In crowd-funding:

- The contract starts in one state,
- Performs actions,
- Then changes to another state.

States in Crowd-Funding Smart Contract

1. Created State

- Smart contract is deployed.
- Funding goal and deadline are defined.

Example:

Goal = ₹1,00,000
Deadline = 30 days

2. Funding State

- Users contribute money to the project.
- Contract records all contributions.

Action: People send funds to the smart contract.

3. Successful State

- Funding goal is reached before deadline.
- Funds are transferred to the project owner.

Condition: Collected Amount ≥ Target Amount

4. Failed State

- Funding goal is not achieved before deadline.
- Contributors can withdraw their money.

Condition: Collected Amount < Target Amount

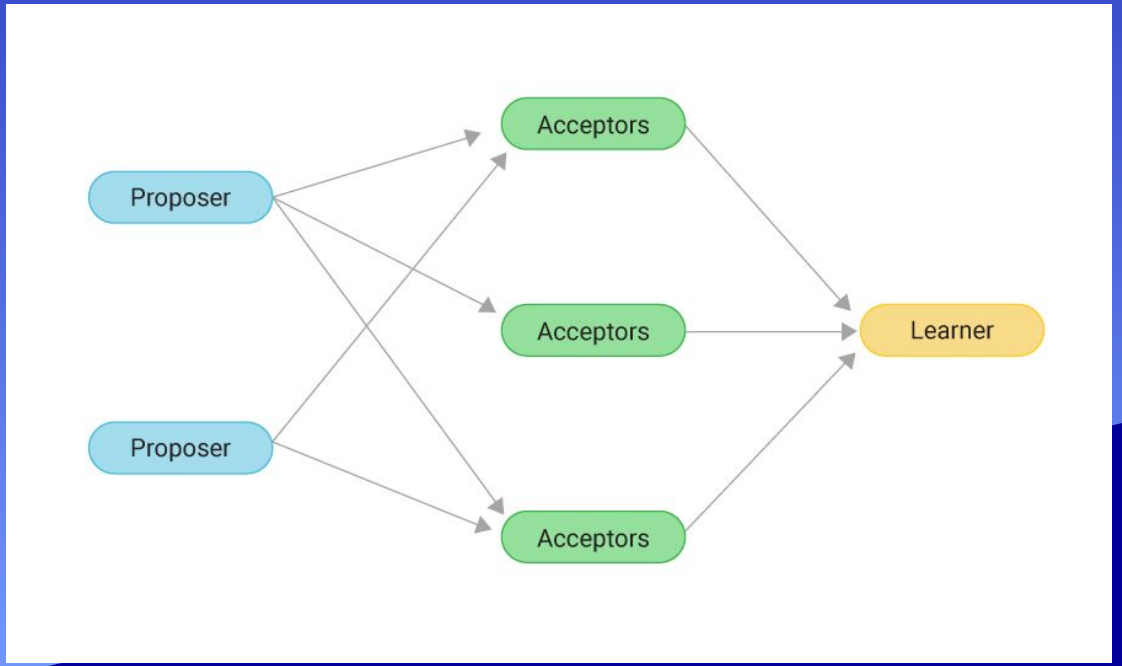
Consensus Algorithms for Private Blockchain

- PAXOS Consensus Algorithm
- Raft Consensus Algorithm



PAXOS Consensus Algorithm

- The PAXOS consensus algorithm is a foundational algorithm in distributed computing. It was designed to solve the consensus problem in the network of asynchronous processes. This article focuses on discussing the PAXOS algorithm in detail.



PAXOS Consensus Algorithm

Introduction: PAXOS is a consensus algorithm used in distributed systems to ensure that multiple computers (nodes) agree on a single value or decision, even if some nodes fail.

It is mainly used in:

- Distributed databases
- Permissioned blockchains
- Fault-tolerant systems
- Cloud computing systems

The algorithm was proposed by Leslie Lamport.

Definition:PAXOS is a consensus protocol that helps distributed nodes agree on the same data value in a reliable and fault-tolerant manner.

Goal of Paxos : The main goal is:

- To achieve agreement among distributed nodes
- Even when some nodes crash or messages are delayed

Roles in Paxos :

1. Proposer : Suggests a value to the network.

2. Acceptor: Receives proposals and decides whether to accept them.

3. Learner:Learns the final agreed value.

Working of Paxos Algorithm:Paxos works in two major phases.

Phase 1: Prepare Phase

Step 1: Proposer sends a Prepare Request with a proposal number to acceptors.

Step 2: Acceptors respond:

- Promise not to accept smaller proposal numbers.
- Send previously accepted proposal if any.

PAXOS Consensus Algorithm

Phase 2: Accept Phase

Step 3: Proposer sends an Accept Request with the value.

Step 4: Acceptors accept the proposal if it follows the promise.

Step 5: Once majority accept:

- Consensus is achieved.
- Learners are informed.

Characteristics of Paxos

- Fault tolerant
- Reliable
- Consistent
- Works in asynchronous networks
- Handles node failures

Advantages

1. High reliability
2. Handles server failures
3. Ensures consistency
4. No single point of failure
5. Suitable for distributed systems

Disadvantages

1. Complex to understand and implement
2. High communication overhead
3. Slower in large networks
4. Difficult debugging

Applications of Paxos

- Distributed databases
- Cloud services
- Distributed file systems
- Permissioned blockchain networks

Explanation of the Paxos Consensus Diagram

The diagram represents the working of the **Paxos Consensus Algorithm** using three main components:

1. **Proposers**
2. **Acceptors**
3. **Learner**

Components in the Diagram

1. Proposer

- A proposer suggests a value or transaction to the network.
- It starts the consensus process.

Example:

"Store Transaction A"

In the diagram:

- Two proposers are shown.
- Multiple proposers can exist in Paxos.

2. Acceptors

- Acceptors receive proposals from proposers.
- They decide whether to accept or reject a proposal.
- Consensus is reached when the majority of acceptors agree.

In the diagram:

- Three acceptors are shown.

3. Learner

- The learner receives the final agreed result from acceptors.
- It learns which proposal was accepted by the majority.

Working of the Diagram

Step 1: Proposal Creation

A proposer creates a proposal and sends it to acceptors.

Proposer → Acceptors

Step 2: Acceptors Process Proposal

Acceptors:

- Check proposal number
- Decide whether to accept it

If valid:

- They accept the proposal.

Step 3: Majority Agreement

If most acceptors agree on the same proposal:

- Consensus is achieved.

Example:

- 2 out of 3 acceptors approve the proposal.

Step 4: Learner Receives Final Value

Acceptors send the accepted value to the learner.

Acceptors → Learner

The learner now knows the final agreed result.

Raft Consensus Algorithm

Introduction: Raft is a consensus algorithm used in distributed systems to help multiple servers (nodes) agree on the same data and maintain consistency.

It is designed to be:

- Simple
- Understandable
- Reliable

Raft is commonly used in:

- Distributed databases
- Permissioned blockchains
- Cloud systems
- Distributed applications

Raft is a consensus protocol that ensures all nodes in a distributed system maintain the same state by electing a leader and replicating logs consistently.

Goal of Raft: The main goals are:

- Maintain consistency among nodes
- Handle node failures
- Ensure reliable data replication
- Simplify distributed consensus

Main Components of Raft

1. Leader

- Controls the system.
- Receives client requests.
- Sends updates to followers.

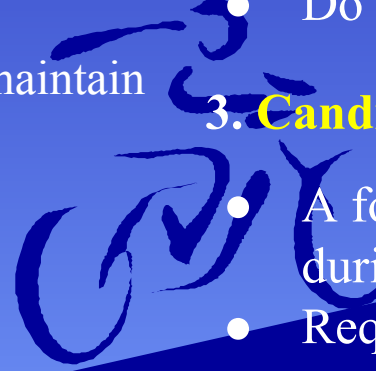
There is only **one leader** at a time.

2. Followers

- Respond to leader requests.
- Store replicated data.
- Do not directly handle client requests.

3. Candidate

- A follower becomes a candidate during leader election.
- Requests votes from other nodes.



Raft Consensus Algorithm

Working of Raft Algorithm: Raft mainly works in three stages.

1. Leader Election

Step 1: All nodes start as followers.

Step 2: If followers do not receive messages from the leader within a timeout: A follower becomes a candidate.

Step 3: Candidate requests votes from other nodes.

Step 4: If the candidate receives majority votes: It becomes the new leader.

2. Log Replication

Step 1: Leader receives client requests.

Step 2: Leader stores the request in its log.

Step 3: Leader sends log entries to followers.

Step 4: Followers confirm receipt.

Step 5: Once majority confirm:

- Entry is committed.
- All nodes update their state.

3. Safety and Consistency

- All nodes execute the same operations
- Logs remain consistent
- Incorrect data is prevented

Features of Raft

- Leader-based consensus
- Easy to understand
- Fault tolerant
- Consistent log replication
- Supports distributed systems

Advantages

1. Simple implementation
2. High consistency
3. Fault tolerance
4. Easy debugging
5. Faster recovery after failure

Disadvantages

1. Leader can become bottleneck
2. Communication overhead
3. Temporary unavailability during leader election

Applications of Raft

- Distributed databases
- Cluster management systems
- Permissioned blockchains
- Cloud platforms

Explain RAFT Consensus algorithm with a suitable example

Components in the Diagram

1. Client

- The **Client** sends requests or transactions to the leader.
- Example:
  - Money transfer request
  - Data update request

2. Leader

- The **Leader** is the main controlling node.
- It receives requests from the client.
- It manages communication with followers.
- Only one leader exists at a time.

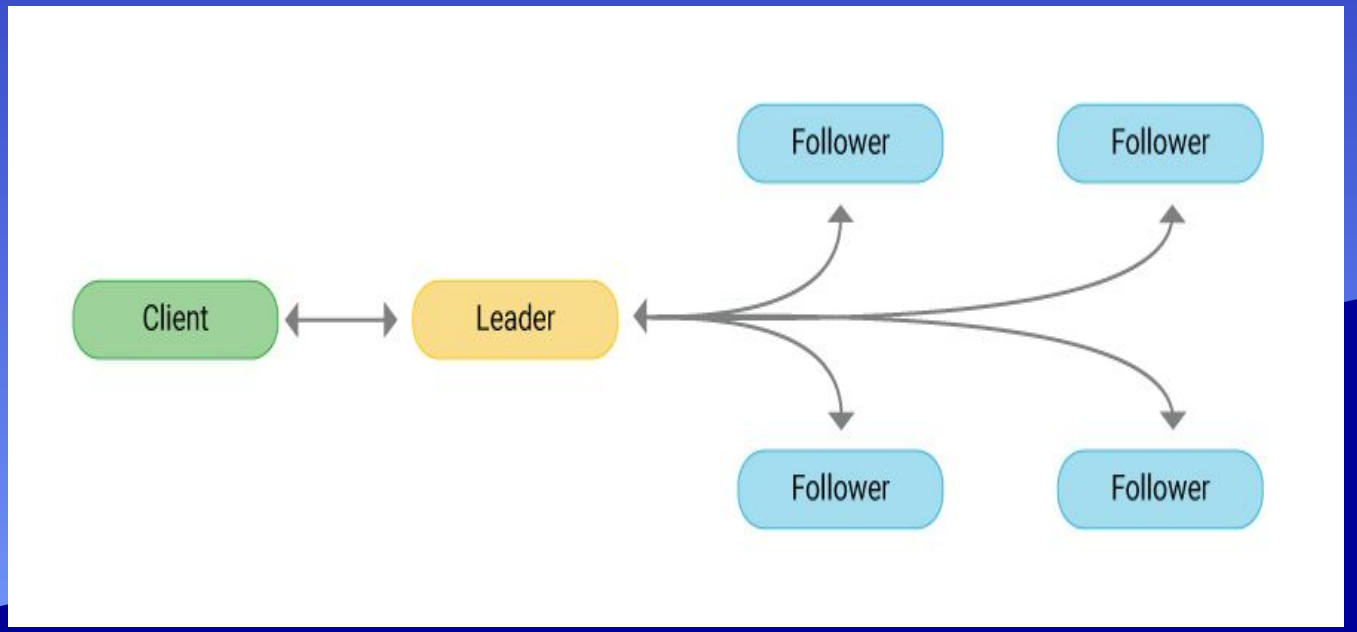
Responsibilities of Leader

- Accept client requests
- Store transaction in log
- Send updates to followers
- Commit transactions after majority approval

3. Followers

- Followers receive updates from the leader.
- They copy and store the same data.
- They help maintain consistency.

There are multiple follower nodes in the network.



Working of the Diagram

Step 1: Client Sends Request: The client sends a transaction request to the leader.

Client → Leader

Example:“Transfer ₹5000”

Step 2: Leader Processes Request: The leader:

- Records the request
- Creates a log entry

Step 3: Leader Sends Data to Followers: The leader replicates the log entry to all follower nodes.

Leader → Followers

This process is called:

Log Replication

Step 4: Followers Confirm Followers acknowledge that they received and stored the log.

Step 5: Transaction Committed If the majority of followers agree:

- The transaction becomes committed.
- All nodes maintain the same data.



Byzantine Faults

- Byzantine Fault Tolerant (BFT)
- Practical BFT



Byzantine Fault Tolerant (BFT)

Byzantine Fault Tolerance (BFT) is a property of distributed systems that allows the system to continue working correctly even when some nodes fail or behave maliciously.

It is used in:

- Blockchain systems
- Distributed networks
- Banking systems
- Cloud computing

BFT helps nodes in a network reach agreement even if some nodes send incorrect or fake information.

A **Byzantine Fault** occurs when: This problem comes from the famous:

- A node fails,
- Sends wrong data,
- Or behaves maliciously.

Byzantine Generals Problem

Imagine several army generals surrounding a city.

They must all agree on:Attack or Retreat

Problems:

- Some generals may be traitors.
- Traitors may send false messages.
- Loyal generals must still reach the same decision.

This situation is called the Byzantine Problem.

Definition of BFT

Byzantine Fault Tolerance is the ability of a distributed system to achieve consensus even if some nodes behave incorrectly or maliciously.

Need for BFT:BFT is needed to:

- Prevent fake transactions
- Maintain consistency
- Handle malicious nodes
- Ensure network reliability
- Improve blockchain security

Working of BFT

Step 1: Transaction Proposal :A node proposes a transaction.

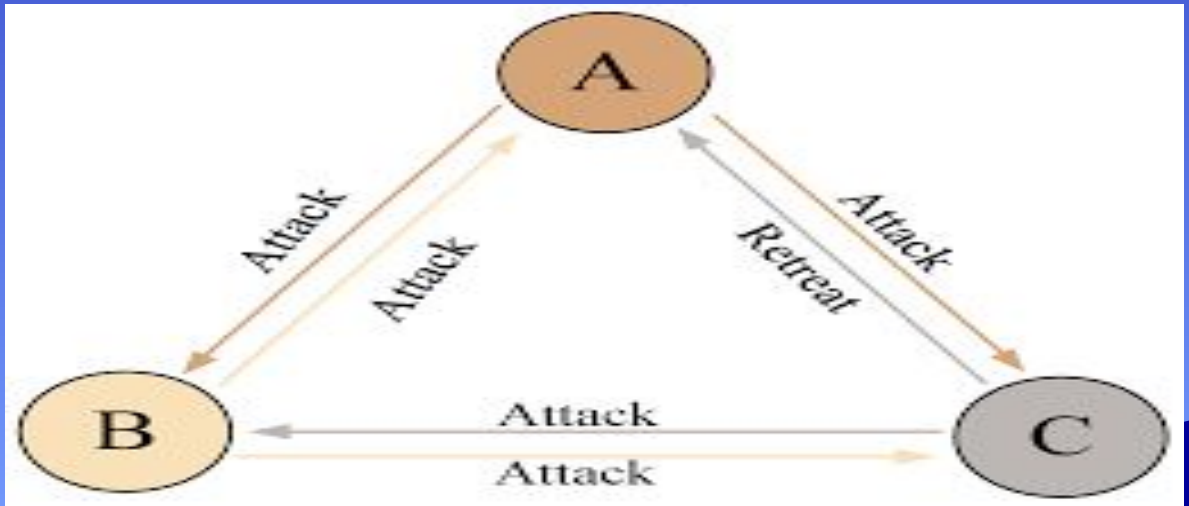
Step 2: Message Sharing: Nodes exchange messages with each other.

Step 3: Verification: Each node verifies the transaction.

Step 4: Voting: Nodes vote to accept or reject the transaction.

Step 5: Consensus: If the majority of honest nodes agree:

- The transaction is accepted.



The diagram represents the **Byzantine Generals Problem**, which is the foundation of **Byzantine Fault Tolerance (BFT)** in distributed systems and blockchain networks.

Components in the Diagram

There are three generals (nodes):

- **A,B,C**

These generals must agree on one common decision:

Attack or Retreat

Meaning of Arrows: The arrows represent messages exchanged between generals.

Examples: "Attack" ,"Retreat"

Each general sends decisions to the others.

Problem Shown in the Diagram: In the diagram:

- General **A** sends different messages to **B** and **C**.
- One message says:

Attack: Another message says:
Retreat:This creates confusion among the generals.

Practical Byzantine Fault Tolerance (PBFT)

Practical Byzantine Fault Tolerance (PBFT) is a consensus algorithm designed to solve the Byzantine Fault Problem in distributed systems.

It allows a network to continue working correctly even if some nodes are faulty or malicious.

PBFT is mainly used in:

- Permissioned blockchains
- Distributed databases
- Enterprise blockchain systems

PBFT is a consensus mechanism that enables distributed nodes to agree on the same transaction even when some nodes behave maliciously.

PBFT is used to:

- Achieve consensus
- Prevent fake transactions
- Handle malicious nodes
- Maintain consistency in distributed systems

Components of PBFT

1. Primary Node (Leader)

- Controls the process.
- Receives client requests.
- Coordinates communication.

2. Replica Nodes (Backup Nodes)

- Verify requests.
- Participate in voting.
- Maintain the same ledger state.

Practical Byzantine Fault Tolerance (PBFT)

Working of PBFT: PBFT mainly works in **three phases**.

Phase 1: Pre-Prepare

Step 1: Client sends request to the primary node.

Step 2: Primary node assigns a sequence number and broadcasts the request to replicas.

Client → Primary → Replicas

Phase 2: Prepare

Step 3: Replica nodes verify the message.

Step 4: Replicas exchange prepare messages with each other.

If enough nodes agree: The request moves to the next phase.

Phase 3: Commit

Step 5: Nodes send commit messages.

Step 6: If majority agreement is achieved:

- Transaction is committed.
- Ledger is updated.

Features of PBFT

- Byzantine fault tolerant
- Fast consensus
- No mining required
- Energy efficient
- High transaction speed

Advantages

1. High security
2. Fast transaction confirmation
3. Handles malicious nodes
4. Energy efficient
5. Suitable for permissioned blockchain



Disadvantages

1. High communication overhead
2. Difficult scalability for large networks
3. Complex implementation

Introduction to Hyperledger

Hyperledger is an open-source blockchain project created to develop enterprise-grade blockchain solutions for businesses and organizations. It was introduced by the Linux Foundation in 2015.

Hyperledger provides tools, frameworks, and libraries for building:

- Private blockchains
- Permissioned blockchain networks
- Business applications

Hyperledger is a collection of open-source blockchain frameworks and tools designed for secure, scalable, and enterprise blockchain applications.

Main Goal of Hyperledger: The main goals are:

- Improve business transactions
- Increase security and transparency
- Reduce cost and intermediaries
- Support enterprise blockchain development

Features of Hyperledger

1. Permissioned Network: Only authorized users can join the network.

2. High Security: Provides secure communication and identity management.

3. Modular Architecture: Different components can be customized as needed.

4. Smart Contract Support: Supports smart contracts (called chaincode in some frameworks).

5. Scalability: Can handle large business transactions efficiently.

6. No Cryptocurrency Requirement: Hyperledger does not require mining or cryptocurrency like Bitcoin.

Hyperledger Architecture

The diagram shows the transaction flow in Hyperledger Fabric Version 1 architecture.

- Clients send transactions,
- Nodes process them,
- And the blockchain ledger gets updated.

1. Client: The client is the user or application interacting with the blockchain.

It starts the transaction process. **Example:**Transfer money

2. Fabric SDK: SDK (Software Development Kit) helps the client communicate with the blockchain network.

Functions

- Sends transaction proposals
- Receives responses
- Connects applications to Fabric

3. Endorser and/or Committer Node: This node performs two major tasks:

a) Endorser Role

- Executes chaincode (smart contract)
- Simulates transaction
- Produces endorsement result

In Diagram: Transaction Proposal → Execution → Response

b) Committer Role

- Validates transactions
- Stores blocks in ledger
- Updates blockchain state

4. World State : World State stores:

- Latest current values of assets
- Current data of the blockchain

Example: Current account balance

It is faster to access than checking the entire blockchain.

Hyperledger Architecture

5. Orderer Node: The Orderer Node:

- Collects endorsed transactions
- Arranges them in correct order
- Creates transaction blocks

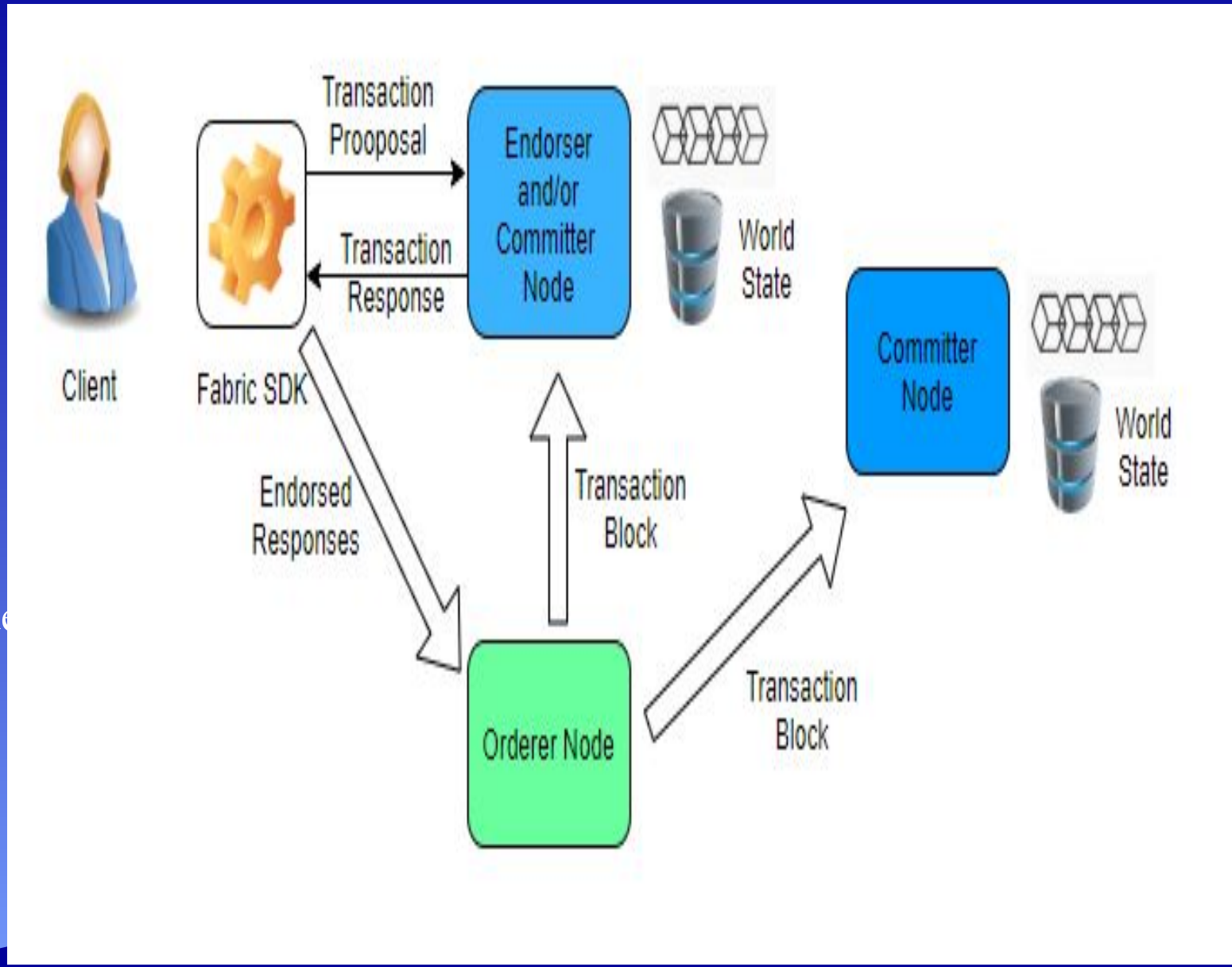
Main Function Transaction Ordering

6. Transaction Block : After ordering:

- Transactions are grouped into blocks.
- Blocks are sent to committee nodes.

7. Committer Node: The Committer Node

- Validates transaction blocks
- Updates ledger and world state
- Stores blockchain data permanently



Hyperledger Architecture

Step 1: Client Sends Proposal

Client uses Fabric SDK to send:Transaction Proposal

to the endorsing node.

Step 2: Endorser Executes Chaincode

The endorsing node:

- Simulates transaction
- Checks smart contract logic
- Generates endorsed response

Step 3: Response Sent to Client

The endorsed transaction response returns to the client.

Endorsed Responses

Step 4: Transaction Sent to Orderer

The client sends endorsed transaction to:

Orderer Node

Step 5: Block Creation

Orderer node:

- Arranges transactions
- Creates transaction blocks

Step 6: Block Sent to Committee Node

The transaction block is distributed to committer nodes.

Step 7: Ledger and World State Updated

Committer node:

- Validates transactions
- Updates blockchain ledger
- Updates world state

Hyperledger Tools

Hyperledger provides various tools that help developers build, test, deploy, and manage blockchain applications.

These tools support:

- Smart contract development
- Network management
- Identity management
- Blockchain monitoring
- Application integration

Important Hyperledger Tools

1. Hyperledger Composer: A tool used to quickly build blockchain business applications.

Features

- Easy smart contract creation
- Business network modeling
- REST API generation
- Rapid application development

Uses

- Supply chain applications
- Asset management systems

2. Hyperledger Explorer

Definition

A web-based blockchain visualization tool.

Features

- View blocks and transactions
- Monitor network activity
- Display smart contract information
- Check node status

Purpose

Helps administrators monitor blockchain networks.

Hyperledger Tools

3. Hyperledger Caliper

Definition: A blockchain performance benchmarking tool.

Features

- Measures transaction speed
- Tests scalability
- Evaluates latency and throughput

Purpose: Used to test blockchain network performance.

4. Hyperledger Cello

Definition: blockchain management and deployment tool.

Features

- Automatic blockchain deployment
- Network management
- Dashboard interface

Purpose : Simplifies blockchain infrastructure setup.

5. Hyperledger Quilt

Definition: A tool for interoperability between different payment systems.

Features

- Cross-network transactions
- Interledger communication

Purpose: Supports payment transfer across multiple blockchain networks.

6. Hyperledger Ursa

Definition: A shared cryptographic library for Hyperledger projects.

Features

- Encryption algorithms
- Digital signatures
- Secure cryptographic operations

Purpose: Provides security functions for blockchain applications.

Hyperledger Tools

7. Hyperledger Avalon

Definition: A framework for trusted off-chain computation.

Features

- Secure computation outside blockchain
- Improved scalability
- Privacy protection

8. Hyperledger Aries

Definition: A tool for decentralized identity management.

Features

- Digital identity creation
- Secure credential exchange
- Authentication support



THANK YOU



BlockChain



Mrs. Pallavi Patil
(Computer Engineering)

Module 6. Tools and Applications of Blockchain

1. Corda, Ripple, Quorum and other Emerging Blockchain Platforms

QBank

1.Write short notes on Corda.

2. Write short notes on Quorum.

3. Write short notes Ripple.



Write short notes on Corda.

Definition: Corda is an open-source, permissioned, distributed ledger platform developed by **R3** for businesses. Unlike public blockchains, it is designed specifically for enterprises in sectors like banking, insurance, and supply chain.

Corda is an open-source distributed ledger platform developed by R3 for businesses. It is mainly designed for industries like banking, finance, insurance, and supply chain management.



Write short notes on Corda.

The diagram shows the main features of Corda.

1. **Identity**
   Every participant in Corda has a verified identity.
   This helps in secure and trusted transactions between organizations.
2. **Consensus**
   Corda uses consensus mechanisms to validate transactions and ensure all parties agree on the transaction data.
3. **Contracts**
   Smart contracts in Corda define business rules and automatically execute agreements between parties.
4. **No Block, But Chain**
   Corda does not store data in traditional blocks like Bitcoin.
   Instead, transactions are connected directly, forming a chain of states.
5. **Privacy**
   Transaction details are shared only with involved parties, ensuring confidentiality and data security.



Write short notes on Corda.

Features of Corda

1. **Permissioned Network:** Only authorized participants can join and access the network.
2. **Privacy and Security:** Transaction data is shared only with parties involved, ensuring confidentiality.
3. **Smart Contracts:** Corda uses smart contracts to automate business agreements and transactions.
4. **No Cryptocurrency Requirement:** Unlike public blockchains, Corda does not require mining or cryptocurrency.
5. **High Performance:** It provides faster transaction processing suitable for enterprise applications.



Write short notes on Corda.

Components of Corda

- **Node** – Represents a participant in the network.
- **Notary** – Prevents double spending and validates transactions.
- **Flow** – Defines the communication process between nodes.
- **Contract** – Contains business rules for transactions.
- **State** – Represents shared facts or data on the ledger.

Advantages of Corda

- Improved security and privacy
- Reduced transaction cost
- Faster business processes
- Better transparency among organizations

Applications of Corda

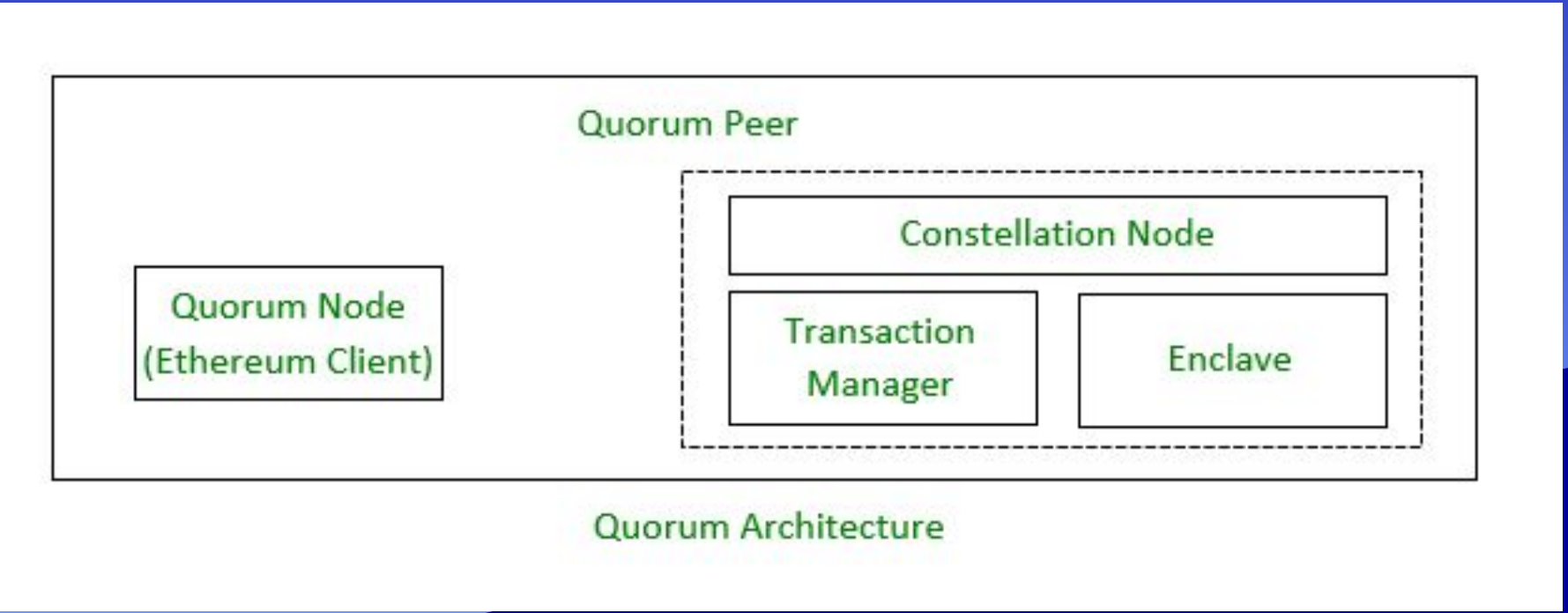
- Banking and payments
- Trade finance
- Insurance claims
- Supply chain tracking
- Digital asset management



Write short notes on Quorum.

Definition: Quorum is an **enterprise-focused, permissioned blockchain platform** developed by **J.P. Morgan**. It is built on top of **Ethereum** to provide businesses with the benefits of blockchain while addressing enterprise requirements like privacy and performance.

Quorum is an enterprise blockchain platform developed by JPMorgan Chase. It is built on the Ethereum blockchain and is specially designed for business and financial applications.



Quorum Architecture

Write short notes on Quorum.

Features of Quorum

1. **Permissioned Network**
   Only authorized participants can access the network.
2. **Privacy of Transactions**
   Quorum supports private transactions where data is shared only with involved parties.
3. **Smart Contracts**
   It uses Ethereum smart contracts to automate business processes.
4. **High Speed and Performance**
   Quorum provides faster transaction processing compared to public Ethereum.
5. **Consensus Mechanisms**
   It uses efficient consensus algorithms like Raft and IBFT instead of mining.



Write short notes on Quorum.

Components of Quorum

- **Node** – A participant in the blockchain network.
- **Smart Contract** – Contains business logic and rules.
- **Consensus Algorithm** – Validates and confirms transactions.
- **Private Transaction Manager** – Ensures transaction privacy.

Advantages

- Secure and private transactions
- Low transaction cost
- Faster processing
- Suitable for enterprise use

Applications

- Banking and finance
- Supply chain management
- Healthcare
- Digital asset management
- Trade finance



Write short notes on Quorum.

1. Quorum Node (Ethereum Client)

- It is the main blockchain node in Quorum.
- Responsible for executing smart contracts and maintaining the distributed ledger.
- Based on the Ethereum client with additional privacy features.

2. Quorum Peer

- A peer represents a participant in the Quorum network.
- Each peer communicates with other peers to validate and share transactions.

3. Constellation Node

- Constellation is a privacy component in Quorum.
- It manages private transactions between authorized participants.
- Ensures confidential data is not visible to all network members.



4. Transaction Manager

- Handles storage and transfer of private transaction data.
- Sends encrypted transaction information only to permitted nodes.

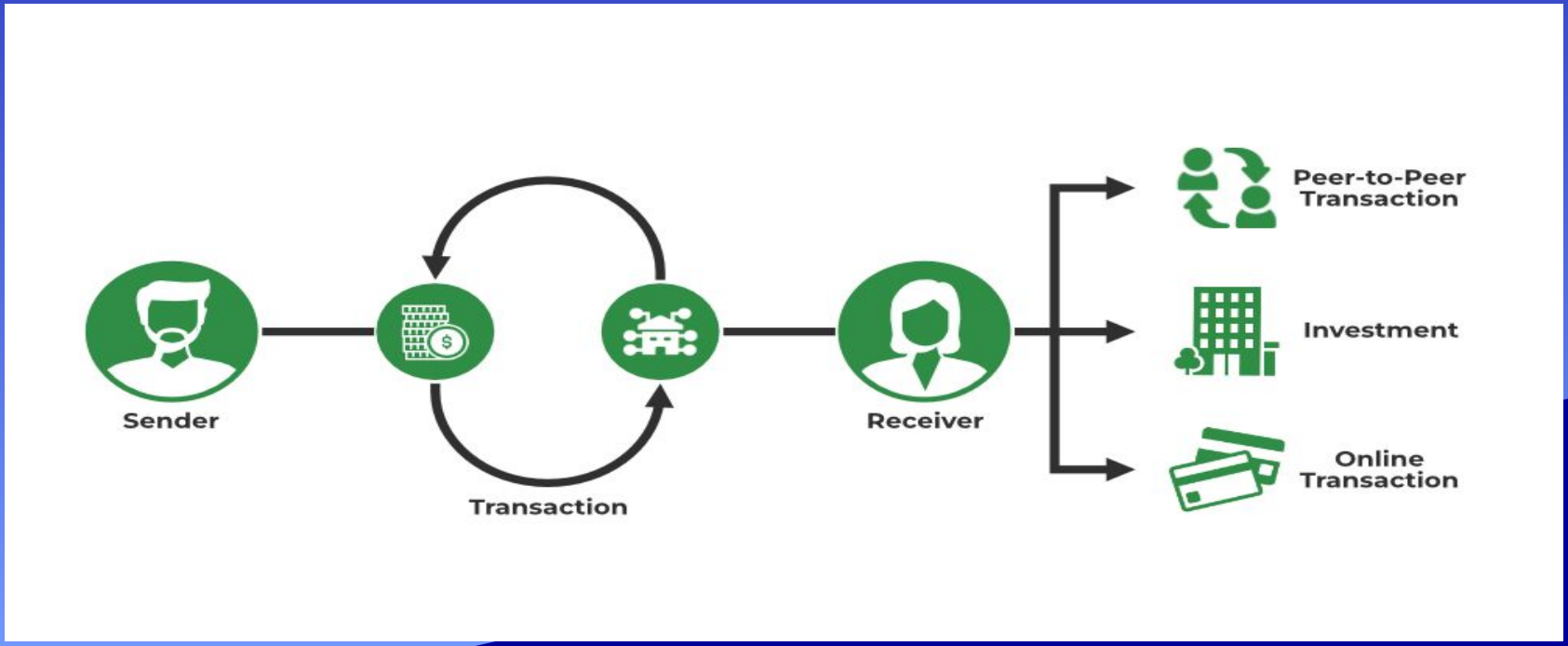
5. Enclave

- Enclave performs encryption and decryption operations.
- It securely manages cryptographic keys and sensitive transaction data.

Write short notes Ripple.

Definition: Ripple is a real-time gross settlement (RTGS) system, currency exchange, and remittance network created by Ripple Labs Inc. It is both a payment protocol and a cryptocurrency (XRP).

Ripple is a blockchain-based digital payment network developed by Ripple Labs. It is mainly designed for fast, secure, and low-cost international money transfers between banks and financial institutions.



Write short notes Ripple.

1. Sender: The process starts when the sender initiates a payment or money transfer using Ripple.

2. Transaction Process : Ripple network verifies and processes the transaction using its consensus protocol. Transactions are completed quickly and securely.

3. Receiver: The receiver gets the transferred amount within a few seconds.

4. Applications of Ripple: The diagram also shows major uses of Ripple:

a) Peer-to-Peer Transaction: Direct money transfer between two individuals without intermediaries.

b) Investment: Ripple and XRP are used in financial investments and digital asset trading.

c) Online Transaction: Ripple supports fast and secure online payments and e-commerce transactions.



Write short notes Ripple.

Features of Ripple

1. **Fast Transactions :** Ripple can process transactions within a few seconds.
2. **Low Transaction Cost:** It offers cheaper international payments compared to traditional banking systems.
3. **Decentralized Network:** Ripple operates on a distributed ledger system.
4. **High Scalability:** It can handle thousands of transactions per second.
5. **Secure Payment System:** Transactions are verified using a consensus protocol.

Components of Ripple

- **RippleNet** – Network used for transferring payments globally.
- **XRP** – Native cryptocurrency of Ripple used for liquidity and transactions.
- **Consensus Ledger** – Maintains and validates transaction records.

Write short notes Ripple.

Advantages

- Faster cross-border payments
- Reduced transaction fees
- Secure and transparent system
- Efficient currency exchange

Applications

- International banking
- Cross-border fund transfers
- Currency exchange systems
- Financial institutions and payment providers



THANK YOU

